

cli.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Cli
4 # =====
5 # Description:
6 #   Консольный интерфейс управления и запросов к базе знаний W
   ikiLLM.
7 #
8 # Usage Examples:
9 #   CLI:
10 #       python -m apps.windows.wikillm.cli
11 #   Python API:
12 #       from apps.windows.wikillm.cli import main
13 #
14 #       res = main()
15 #
16 # File: cli.py
17 # Project: ai-breadboard
18 # Package: apps.windows.wikillm
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-01 13:28:28
22 # =====
23
24 from __future__ import annotations
25 """Консольный интерфейс управления и запросов к базе знаний Wi
   kiLLM."""
26
27 import argparse
28 import asyncio
29 import json
30 import sys
31 from pathlib import Path
```

```

32 from .code_indexer import CodeKnowledgeIndexer
33 from .engine import WikiLLMEngine
34 from .extractor import ArtifactExtractor
35 from .models import ArtifactInput
36
37
38 async def _async_cli_main(args: argparse.Namespace) -> None:
39     """Асинхронная точка входа для командной строки."""
40     engine = WikiLLMEngine()
41
42     if args.command == "resolve":
43         extracted = ArtifactExtractor.from_raw_text(args.query
44 )
45         art = extracted[0] if extracted else ArtifactInput(raw
46 _query=args.query)
47         print(f"[*] Разрешение артефакта: {args.query}...")
48         res = await engine.resolve(art, sync_gemini=not args.n
49 o_llm)
50         print(f"\n[+] Результат ({res.lookup_level.value}, cac
51 hed={res.cached}, {res.execution_time_ms:.1f}ms):")
52         if res.entity:
53             print(f"    Ключ:          {res.entity.canonical_key}"
54 )
55             print(f"    Имя:          {res.entity.name}")
56             print(f"    Категория:    {res.entity.category} ({res
57 .entity.severity})")
58             print(f"    Сводка:      {res.entity.summary}")
59             if res.entity.diagnostic_info:
60                 print(f"    Причины:      {'', ' '.join(res.entity.d
61 iagnostic_info.possible_causes)}")
62                 if res.entity.diagnostic_info.remediation_step
63 s:
64                     print("    Шаги устранения:")
65                     for s in res.entity.diagnostic_info.remedi
66 ation_steps:
67                         print(f"        • {s.title}: {s.descriptio
68 n}")
69             else:

```

```

60         print(f" [-] {res.message}")
61
62     elif args.command == "search":
63         results = engine.storage.search_fts(args.query, limit=
args.limit)
64         print(f"[+] Найдено результатов: {len(results)}")
65         for ent in results:
66             print(f" • [{ent.canonical_key}] {ent.name} - {en
t.summary[:80]}...")
67
68     elif args.command == "index-code":
69         indexer = CodeKnowledgeIndexer(engine.storage)
70         print(f"[*] Индексация исходного кода из {args.path}..
.")
71         count = indexer.index_directory(args.path)
72         print(f"[+] Успешно проиндексировано {count} символов.
")
73
74     elif args.command == "stats":
75         metrics = engine.get_metrics()
76         print(json.dumps(metrics, indent=2, ensure_ascii=False
))
77
78     elif args.command == "tui":
79         from .tui import run_tui
80         await run_tui(engine)
81
82     else:
83         print("Неизвестная команда. Используйте --help для спр
авки.")
84
85
86 def main() -> None:
87     """Главная синхронная точка входа CLI."""
88     parser = argparse.ArgumentParser(description="WikiLLM: Pro
gressive Knowledge Base CLI")
89     subparsers = parser.add_subparsers(dest="command", help="K
оманды")

```

```
90
91     # resolve
92     p_res = subparsers.add_parser("resolve", help="Разрешить о
| ошибку, Event ID или симптом")
93     p_res.add_argument("query", help="Код ошибки, Event ID или
| сообщение")
94     p_res.add_argument("--no-llm", action="store_true", help="
| Не вызывать Gemini при промахе кэша")
95
96     # search
97     p_search = subparsers.add_parser("search", help="Полнотекс
| товый поиск в базе знаний")
98     p_search.add_argument("query", help="Поисковая фраза")
99     p_search.add_argument("--limit", type=int, default=10, hel
| p="Лимит результатов")
100
101     # index-code
102     p_idx = subparsers.add_parser("index-code", help="Индексац
| ия кода в слой Code Knowledge")
103     p_idx.add_argument("--path", default="apps/windows", help=
| "Путь к каталогу с кодом")
104
105     # stats
106     subparsers.add_parser("stats", help="Статистика базы знани
| й и эффективность кэша")
107
108     # tui
109     subparsers.add_parser("tui", help="Запуск интерактивного T
| UI терминала")
110
111     args = parser.parse_args()
112     if not args.command:
113         parser.print_help()
114         sys.exit(0)
115
116     asyncio.run(_async_cli_main(args))
117
118
```

```
119 | if __name__ == "__main__":  
120 |     main()  
121 |
```

code_indexer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Code Indexer
4 # =====
5 # Description:
6 #   Индексатор исходного кода на базе Python AST. Извлекает функции, классы,
7 #
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.wikillm.code_indexer import CodeKnowledgeIndexer
11 #
12 #   service = CodeKnowledgeIndexer()
13 #
14 # File: code_indexer.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Индексатор исходного кода на базе Python AST. Извлекает функции, классы, """
24
25 import ast
26 from pathlib import Path
27 from typing import Any, Dict, List, Optional
28 from logger import logger
29 from .models import (
```

```

30 |     ArtifactType,
31 |     CodeKnowledge,
32 |     KnowledgeEntity,
33 |     KnowledgeSource,
34 |     utc_now_iso,
35 | )
36 | from .storage import WikiStorage
37 |
38 |
39 | class CodeKnowledgeIndexer:
40 |     """Индексатор структуры Python файлов для слоя Code Knowledge."""
41 |
42 |     def __init__(self, storage: WikiStorage) -> None:
43 |         """Инициализирует индексатор кода.
44 |
45 |         Args:
46 |             storage: Экземпляр хранилища WikiStorage.
47 |         """
48 |         self.storage = storage
49 |
50 |     def index_file(self, file_path: Path | str, base_dir: Optional[Path | str] = None) -> List[KnowledgeEntity]:
51 |         """Индексирует отдельный Python файл и сохраняет обнаруженные символы в БД.
52 |
53 |         Args:
54 |             file_path: Путь к Python файлу.
55 |             base_dir: Базовая директория для вычисления относительного пути модуля.
56 |
57 |         Returns:
58 |             Список созданных сущностей KnowledgeEntity.
59 |         """
60 |         p = Path(file_path).resolve()
61 |         if not p.exists() or p.suffix != ".py":
62 |             return []
63 |

```

```

64         try:
65             content = p.read_text(encoding="utf-8", errors="replace")
66             tree = ast.parse(content, filename=str(p))
67         except Exception as exc:
68             logger.warning(f"Ошибка парсинга AST файла {p}: {exc}")
69             return []
70
71         rel_module = p.name
72         if base_dir:
73             try:
74                 rel_module = str(p.relative_to(Path(base_dir).resolve())).replace("\\", "/").replace(".py", "")
75             except ValueError:
76                 rel_module = p.stem
77
78         entities: List[KnowledgeEntity] = []
79         imports: List[str] = []
80
81         # 1. Сбор импортов
82         for node in ast.walk(tree):
83             if isinstance(node, ast.Import):
84                 for alias in node.names:
85                     imports.append(alias.name)
86             elif isinstance(node, ast.ImportFrom) and node.module:
87                 imports.append(node.module)
88
89         # 2. Обход функций и классов верхнего уровня
90         for node in tree.body:
91             if isinstance(node, ast.ClassDef):
92                 ent = self._process_class_node(node, rel_module, imports)
93                 if ent:
94                     entities.append(ent)
95                     self.storage.save_entity(ent)
96

```

```

97         # Методы класса
98         for item in node.body:
99             if isinstance(item, (ast.FunctionDef, ast.
AsyncFunctionDef)):
100                 m_ent = self._process_function_node(
101                     item, rel_module, imports, parent_
class=node.name
102                 )
103                 if m_ent:
104                     entities.append(m_ent)
105                     self.storage.save_entity(m_ent)
106
107             elif isinstance(node, (ast.FunctionDef, ast.AsyncF
unctionDef)):
108                 ent = self._process_function_node(node, rel_mo
odule, imports)
109                 if ent:
110                     entities.append(ent)
111                     self.storage.save_entity(ent)
112
113             return entities
114
115     def _process_class_node(
116         self, node: ast.ClassDef, module_path: str, dependenci
es: List[str]
117     ) -> Optional[KnowledgeEntity]:
118         """Формирует сущность базы знаний для класса."""
119         doc = ast.get_docstring(node) or ""
120         canonical_key = f"code:{module_path}:{node.name}"
121         summary = doc.split("\n\n")[0].strip() if doc else f"K
ласс {node.name}"
122
123         code_info = CodeKnowledge(
124             module_path=module_path,
125             symbol_name=node.name,
126             symbol_type="class",
127             docstring=doc,
128             dependencies=dependencies[:15],

```

```

129         )
130
131     return KnowledgeEntity(
132         canonical_key=canonical_key,
133         entity_type=ArtifactType.CODE_SYMBOL,
134         name=node.name,
135         summary=summary[:250],
136         category="code",
137         severity="info",
138         confidence=1.0,
139         provenance_source=KnowledgeSource.DOCUMENTED,
140         code_info=code_info,
141         tags=["code", "class", module_path.split("/")[-1]]
142     ),
143
144     def _process_function_node(
145         self,
146         node: ast.FunctionDef | ast.AsyncFunctionDef,
147         module_path: str,
148         dependencies: List[str],
149         parent_class: Optional[str] = None,
150     ) -> Optional[KnowledgeEntity]:
151         """Формирует сущность базы знаний для функции или мето
да."""
152         doc = ast.get_docstring(node) or ""
153         full_name = f"{parent_class}.{node.name}" if parent_cl
ass else node.name
154         canonical_key = f"code:{module_path}:{full_name}"
155         summary = doc.split("\n\n")[0].strip() if doc else f"Ф
ункция {full_name}"
156
157         params = [arg.arg for arg in node.args.args if arg.arg
!= "self"]
158         ret_type = ast.unparse(node.returns) if node.returns e
lse None
159
160         code_info = CodeKnowledge(

```

```

161         module_path=module_path,
162         symbol_name=full_name,
163         symbol_type="method" if parent_class else "functio
n",
164         docstring=doc,
165         parameters=params,
166         return_type=ret_type,
167         dependencies=dependencies[:15],
168     )
169
170     return KnowledgeEntity(
171         canonical_key=canonical_key,
172         entity_type=ArtifactType.CODE_SYMBOL,
173         name=full_name,
174         summary=summary[:250],
175         category="code",
176         severity="info",
177         confidence=1.0,
178         provenance_source=KnowledgeSource.DOCUMENTED,
179         code_info=code_info,
180         tags=["code", "function", module_path.split("/")[-
1]],
181     )
182
183     def index_directory(self, dir_path: Path | str) -> int:
184         """Рекурсивно индексирует все .py файлы в каталоге.
185
186         Args:
187             dir_path: Путь к директории исходного кода.
188
189         Returns:
190             Общее количество проиндексированных сущностей.
191         """
192         p = Path(dir_path).resolve()
193         if not p.exists() or not p.is_dir():
194             return 0
195
196         count = 0

```

```
197 |         for py_file in p.glob("**/*.py"):
198 |             if any(part.startswith(".") or part.startswith("~"
199 | ) or part == "__pycache__" for part in py_file.parts):
200 |                 continue
201 |                 ents = self.index_file(py_file, base_dir=p)
202 |                 count += len(ents)
203 |
204 |         return count
```

config.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Config
4 # =====
5 # Description:
6 #   Загрузка и валидация конфигурационных параметров для подсистемы WikiLLM.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.wikillm.config import WikiLLMConfig
11 #
12 #       service = WikiLLMConfig()
13 #
14 # File: config.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Загрузка и валидация конфигурационных параметров для подсистемы WikiLLM."""
24
25 import json
26 import os
27 from pathlib import Path
28 from typing import Any, Dict
29 from pydantic import BaseModel, Field
30
31
```

```
32 class WikiLLMConfig(BaseModel):
33     """Модель конфигурации WikiLLM."""
34
35     database_path: str = Field(
36         default="data/windows_wikillm/knowledge.db",
37         description="Путь к SQLite базе знаний WikiLLM",
38     )
39     min_confidence_threshold: float = Field(
40         default=0.70,
41         description="Минимальный порог уверенности для фиксац  
и фактов",
42     )
43     max_queue_size: int = Field(
44         default=1000,
45         description="Максимальный размер асинхронной очереди н  
еизвестных артефактов",
46     )
47     async_workers: int = Field(
48         default=2,
49         description="Количество параллельных воркеров разреше  
ния артефактов",
50     )
51     gemini_model_id: str = Field(
52         default="gemini-flash-latest",
53         description="Идентификатор модели Gemini для разреше  
ния неизвестных сущностей",
54     )
55     enable_code_indexer: bool = Field(
56         default=True,
57         description="Флаг включения индексатора исходного кода  
кодовой базы",
58     )
59     enable_semantic_search: bool = Field(
60         default=True,
61         description="Флаг включения семантического поиска",
62     )
63     cache_exact_hits: bool = Field(
64         default=True,
```

```

65         description="Кэшировать ли точные совпадения в оперативной памяти",
66     )
67
68
69 def load_config(config_path: Path | str | None = None) -> WikiLLMConfig:
70     """Загружает конфигурацию WikiLLM из JSON файла или возвращает значения по умолчанию.
71
72     Args:
73         config_path: Путь к файлу конфигурации (опционально).
74
75     Returns:
76         Экземпляр WikiLLMConfig.
77     """
78     default_cfg_path = Path(__file__).parent / "config.json"
79     target_path = Path(config_path) if config_path else default_cfg_path
80
81     if target_path.exists() and target_path.is_file():
82         try:
83             with open(target_path, "r", encoding="utf-8") as f:
84                 data: Dict[str, Any] = json.load(f)
85                 return WikiLLMConfig(**data)
86         except Exception:
87             pass
88
89     return WikiLLMConfig()
90

```

engine.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Engine
4 # =====
5 # Description:
6 #   Главный координатор прогрессивной базы знаний WikiLLM. Упр
   авляет 4-уровневым
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.wikillm.engine import WikiLLMEngine
11 #
12 #       service = WikiLLMEngine()
13 #
14 # File: engine.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Главный координатор прогрессивной базы знаний WikiLLM. Упр
   авляет 4-уровневым"""
24
25 import asyncio
26 import time
27 from typing import Any, Dict, List, Optional
28 from logger import logger
29 from .config import WikiLLMConfig, load_config
30 from .models import (
31     ArtifactInput,
```

```

32     KnowledgeEntity,
33     LookupLevel,
34     ResolutionResult,
35 )
36 from .normalizer import CanonicalKeyNormalizer
37 from .resolvers.exact import ExactResolver
38 from .resolvers.fingerprint import FingerprintResolver
39 from .resolvers.gemini import GeminiKnowledgeResolver
40 from .resolvers.semantic import SemanticResolver
41 from .storage import WikiStorage
42
43
44 class WikiLLMEngine:
45     """Движок прогрессивного накопления знаний и 4-уровневой д
    испетчеризации запросов."""
46
47     def __init__(
48         self,
49         config: Optional[WikiLLMConfig] = None,
50         storage: Optional[WikiStorage] = None,
51         chat_model: Optional[Any] = None,
52     ) -> None:
53         """Инициализирует WikiLLMEngine.
54
55         Args:
56             config: Конфигурация WikiLLMConfig или None для ав
    томатической загрузки.
57             storage: Экземпляр хранилища WikiStorage.
58             chat_model: Экземпляр модели Gemini Chat.
59         """
60         self.config = config or load_config()
61         self.storage = storage or WikiStorage(self.config.data
    base_path)
62
63         # Инициализация 4 уровней резолверов
64         self.exact_resolver = ExactResolver(self.storage)
65         self.fingerprint_resolver = FingerprintResolver(self.s
    torage)

```

```

66         self.semantic_resolver = SemanticResolver(self.storage
67     )
68     self.gemini_resolver = GeminiKnowledgeResolver(
69         chat_model=chat_model,
70         model_id=self.config.gemini_model_id,
71     )
72     # Очередь для асинхронного разрешения артефактов телем
етрии
73     self._async_queue: asyncio.Queue[ArtifactInput] = asyn
cio.Queue(
74         maxsize=self.config.max_queue_size
75     )
76     self._worker_tasks: List[asyncio.Task] = []
77     self._is_running = False
78
79     # Метрики и статистика поиска
80     self._stats = {
81         "total_requests": 0,
82         "l1_exact_hits": 0,
83         "l2_fingerprint_hits": 0,
84         "l3_semantic_hits": 0,
85         "l4_gemini_resolutions": 0,
86         "unresolved_count": 0,
87     }
88
89     async def start(self) -> None:
90         """Запускает фоновые воркеры асинхронной очереди разре
шения артефактов."""
91         if self._is_running:
92             return
93         self._is_running = True
94         for i in range(max(1, self.config.async_workers)):
95             task = asyncio.create_task(self._queue_worker_loop
(i), name=f"WikiLLM-Worker-{i}")
96             self._worker_tasks.append(task)
97             logger.info(f"WikiLLMEngine запущен с {len(self._worke
r_tasks)} фоновыми воркерами.")

```

```

98
99     async def stop(self) -> None:
100         """Останавливает фоновые воркеры."""
101         self._is_running = False
102         for task in self._worker_tasks:
103             task.cancel()
104         self._worker_tasks.clear()
105         logger.info("WikiLLMEngine остановлен.")
106
107     async def _queue_worker_loop(self, worker_id: int) -> None
108     :
109         """Фоновый цикл обработки артефактов из очереди."""
110         while self._is_running:
111             try:
112                 artifact = await self._async_queue.get()
113                 canonical_key = CanonicalKeyNormalizer.compute
114                 _canonical_key(artifact)
115                 # Проверяем, не разрешен ли уже артефакт
116                 existing = self.storage.get_entity(canonical_k
117                 ey)
118                 if not existing:
119                     logger.debug(f"[Worker-{worker_id}] Асинхр
120                     онное разрешение через Gemini: {canonical_key}")
121                     entity = await self.gemini_resolver.resolv
122                     e(artifact)
123                     if entity:
124                         self.storage.save_entity(entity)
125                         self._stats["l4_gemini_resolutions"] +
126                         = 1
127                         logger.info(f"[Worker-{worker_id}] Арт
128                         ефакт {canonical_key} успешно сохранен в базе знаний.")
129                         self._async_queue.task_done()
130                     except asyncio.CancelledError:
131                         break
132                     except Exception as exc:
133                         logger.warning(f"[Worker-{worker_id}] Ошибка о
134                         бработки артефакта из очереди: {exc}")
135                         await asyncio.sleep(1.0)

```

```

128
129     async def resolve(
130         self,
131         artifact: ArtifactInput,
132         sync_gemini: bool = True,
133         record_observation: bool = True,
134         co_occurring_keys: Optional[List[str]] = None,
135     ) -> ResolutionResult:
136         """Главный метод разрешения артефакта через 4-уровневый конвейер.
137
138         Args:
139             artifact: Входной артефакт (код ошибки, Event ID, процесс, симптом).
140             sync_gemini: Разрешать ли неизвестные артефакты синхронно через Gemini.
141             record_observation: Фиксировать ли наблюдение в счетчиках локальной машины.
142             co_occurring_keys: Список ключей сопутствующих событий для графа связей.
143
144         Returns:
145             Результат ResolutionResult.
146         """
147         start_time = time.perf_counter()
148         self._stats["total_requests"] += 1
149         canonical_key = CanonicalKeyNormalizer.compute_canonical_key(artifact)
150
151         # Фиксация наблюдения
152         if record_observation and canonical_key and canonical_key != "unknown:unspecified":
153             self.storage.record_observation(canonical_key, co_occurring_keys=co_occurring_keys)
154
155         # Level 1: Exact Match (SQLite O(1))
156         entity = await self.exact_resolver.resolve(artifact)
157         if entity:

```

```

158         self._stats["l1_exact_hits"] += 1
159         elapsed_ms = (time.perf_counter() - start_time) *
160         1000.0
161         return ResolutionResult(
162             canonical_key=canonical_key,
163             lookup_level=LookupLevel.EXACT,
164             cached=True,
165             entity=entity,
166             confidence=entity.confidence,
167             execution_time_ms=elapsed_ms,
168             message="Мгновенный ответ из локальной базы зн
аний (Level 1 Exact)",
169         )
170
171     # Level 2: Fingerprint Template Match
172     entity = await self.fingerprint_resolver.resolve(artif
act)
173
174     if entity:
175         self._stats["l2_fingerprint_hits"] += 1
176         elapsed_ms = (time.perf_counter() - start_time) *
177         1000.0
178         return ResolutionResult(
179             canonical_key=canonical_key,
180             lookup_level=LookupLevel.FINGERPRINT,
181             cached=True,
182             entity=entity,
183             confidence=entity.confidence * 0.95,
184             execution_time_ms=elapsed_ms,
185             message="Совпадение по структурному шаблону (L
evel 2 Fingerprint)",
186         )
187
188     # Level 3: Semantic / FTS5 Match
189     if self.config.enable_semantic_search:
190         entity = await self.semantic_resolver.resolve(arti
fact)
191
192         if entity:
193             self._stats["l3_semantic_hits"] += 1

```

```

190         elapsed_ms = (time.perf_counter() - start_time
191         ) * 1000.0
192         return ResolutionResult(
193             canonical_key=canonical_key,
194             lookup_level=LookupLevel.SEMANTIC,
195             cached=True,
196             entity=entity,
197             confidence=entity.confidence * 0.85,
198             execution_time_ms=elapsed_ms,
199             message="Найдено по семантическому сходств
200 y FTS5 (Level 3 Semantic)",
201         )
202
203     # Level 4: Gemini Structured Resolver
204     if sync_gemini:
205         entity = await self.gemini_resolver.resolve(artifa
206 ct)
207         if entity:
208             # Валидация и самообучение: сохранение в SQLit
209 e
210             self.storage.save_entity(entity)
211             self._stats["l4_gemini_resolutions"] += 1
212             elapsed_ms = (time.perf_counter() - start_time
213             ) * 1000.0
214             return ResolutionResult(
215                 canonical_key=canonical_key,
216                 lookup_level=LookupLevel.GEMINI,
217                 cached=False,
218                 entity=entity,
219                 confidence=entity.confidence,
220                 execution_time_ms=elapsed_ms,
221                 message="Новое знание сгенерировано через
222 Gemini и сохранено в SQLite (Level 4)",
223             )
224
225     # Если sync_gemini = False, отправляем в очередь асинх
226 ронного обучения
227     if not sync_gemini and self._is_running:

```

```

221         try:
222             self._async_queue.put_nowait(artifact)
223         except asyncio.QueueFull:
224             logger.warning("Очередь WikiLLM переполнена, а
ртефакт пропущен.")
225
226         self._stats["unresolved_count"] += 1
227         elapsed_ms = (time.perf_counter() - start_time) * 1000
228         .0
229         return ResolutionResult(
230             canonical_key=canonical_key,
231             lookup_level=LookupLevel.UNKNOWN,
232             cached=False,
233             entity=None,
234             confidence=0.0,
235             execution_time_ms=elapsed_ms,
236             message="Артефакт не найден и добавлен в очередь а
синхронного разрешения"
237             if not sync_gemini
238             else "Не удалось разрешить артефакт",
239         )
240
241     def get_metrics(self) -> Dict[str, Any]:
242         """Возвращает метрики производительности и эффективнос
ть кэша."""
243         total = self._stats["total_requests"]
244         hits = (
245             self._stats["l1_exact_hits"]
246             + self._stats["l2_fingerprint_hits"]
247             + self._stats["l3_semantic_hits"]
248         )
249         hit_rate = (hits / total * 100.0) if total > 0 else 0.
250         0
251         db_stats = self.storage.get_stats()
252         return {
253             "lookup_stats": self._stats,
254             "cache_hit_rate_pct": round(hit_rate, 2),

```

```
254 |         "queue_size": self._async_queue.qsize(),
255 |         "storage_stats": db_stats,
256 |     }
257 |
```

extractor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Extractor
4 # =====
5 # Description:
6 #   Извлечение типизированных артефактов (ArtifactInput) из телеметрии, инцидентов,
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.wikillm.extractor import ArtifactExtractor
11 #
12 #       service = ArtifactExtractor()
13 #
14 # File: extractor.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Извлечение типизированных артефактов (ArtifactInput) из телеметрии, инцидентов, """
24
25 import re
26 from typing import Any, Dict, List, Optional
27 from .models import ArtifactInput, ArtifactType
28 from .normalizer import CanonicalKeyNormalizer
29
30
```

```

31 class ArtifactExtractor:
32     """Извлекает нормализованные входные артефакты из сырых да
    нных и логов."""
33
34     _ERROR_CODE_REGEX = re.compile(r"\b(0x[0-9a-fA-F]{8}|0x[0-
    9a-fA-F]{4})\b")
35     _EVENT_ID_REGEX = re.compile(r"\b(?:EventID|Event ID|Event
    |ID)[:\s]+(\d{1,6})\b", re.IGNORECASE)
36     _REGISTRY_REGEX = re.compile(r"\b(HKLM\\[a-zA-Z0-9_\\-]+|
    HKCU\\[a-zA-Z0-9_\\-]+|HKEY_[A-Z_]+\\[a-zA-Z0-9_\\-]+)\b")
37     _PROCESS_REGEX = re.compile(r"\b([a-zA-Z0-9_\\-\\.]+\\.?(?:exe
    |dll|sys))\b", re.IGNORECASE)
38
39     @classmethod
40     def from_event_dict(cls, event: Dict[str, Any]) -> Artificac
    tInput:
41         """Извлекает артефакт из словаря события Windows Event
    Log.
42
43         Args:
44             event: Словарь с полями события (Provider, EventID
    , Message, etc.).
45
46         Returns:
47             Экземпляр ArtifactInput.
48         """
49         provider = str(event.get("Provider") or event.get("pro
    vider") or event.get("source") or "Generic")
50         raw_id = event.get("EventID") or event.get("event_id")
    or event.get("id")
51         event_id: Optional[int] = None
52         if raw_id is not None:
53             try:
54                 event_id = int(raw_id)
55             except (ValueError, TypeError):
56                 event_id = None
57
58         message = str(event.get("Message") or event.get("messa

```

```

    ge") or event.get("description") or "")
59     process_name = str(event.get("ProcessName") or event.g
    et("process") or "") or None
60
61     return ArtifactInput(
62         type=ArtifactType.WINDOWS_EVENT,
63         provider=provider,
64         event_id=event_id,
65         process_name=process_name,
66         message=message,
67         metadata=event,
68     )
69
70     @classmethod
71     def from_incident(cls, incident_data: Dict[str, Any]) -> L
    ist[ArtifactInput]:
72         """Извлекает артефакты из объекта или словаря инцидент
    a Telemetry Engine.
73
74         Args:
75             incident_data: Словарь инцидента (category, trigge
    r_type, suspects, error_code).
76
77         Returns:
78             Список извлеченных артефактов ArtifactInput.
79         """
80         artifacts: List[ArtifactInput] = []
81
82         # Базовый артефакт инцидента
83         cat = incident_data.get("category", "system")
84         trigger = incident_data.get("trigger_type", "anomaly")
85         artifacts.append(
86             ArtifactInput(
87                 type=ArtifactType.INCIDENT,
88                 raw_query=f"{cat}_{trigger}",
89                 message=incident_data.get("description", ""),
90                 metadata=incident_data,
91             )

```

```

92         )
93
94         # Извлечение подозреваемых процессов
95         suspects = incident_data.get("suspects") or incident_d
125 ata.get("suspect_processes") or []
96         for s in suspects:
97             p_name = s.get("name") or s.get("process_name") if
125 isinstance(s, dict) else str(s)
98             if p_name:
99                 artifacts.append(
125                     ArtifactInput(
126                         type=ArtifactType.PROCESS,
127                         process_name=p_name,
128                         metadata={"suspect_info": s},
129                     )
130                 )
131
132         # Извлечение кода ошибки при наличии
133         err = incident_data.get("error_code") or incident_data
134         .get("hresult")
135         if err:
136             artifacts.append(
137                 ArtifactInput(
138                     type=ArtifactType.WINDOWS_ERROR,
139                     error_code=str(err),
140                     message=incident_data.get("title", ""),
141                 )
142             )
143
144         return artifacts
145
146     @classmethod
147     def from_raw_text(cls, text: str) -> List[ArtifactInput]:
148         """Извлекает распознанные артефакты (ошибки, события,
149 процессы, реестр) из свободного текста.
150
151     Args:
152         text: Произвольный текст лога, сообщения об ошибке

```

или диагностического отчета.

```
126
127     Returns:
128         Список обнаруженных артефактов ArtifactInput.
129     """
130     if not text:
131         return []
132
133     results: List[ArtifactInput] = []
134     seen_keys = set()
135
136     # 1. Поиск кодов ошибок (0x80070490, 0xc0000005)
137     for match in cls._ERROR_CODE_REGEX.finditer(text):
138         code = match.group(1)
139         art = ArtifactInput(
140             type=ArtifactType.WINDOWS_ERROR,
141             error_code=code,
142             message=text[:200],
143         )
144         key = CanonicalKeyNormalizer.compute_canonical_key
145         (art)
146         if key not in seen_keys:
147             seen_keys.add(key)
148             results.append(art)
149
150     # 2. Поиск Event ID
151     for match in cls._EVENT_ID_REGEX.finditer(text):
152         eid_str = match.group(1)
153         try:
154             eid = int(eid_str)
155             art = ArtifactInput(
156                 type=ArtifactType.WINDOWS_EVENT,
157                 event_id=eid,
158                 message=text[:200],
159             )
160             key = CanonicalKeyNormalizer.compute_canonical
161             _key(art)
162             if key not in seen_keys:
```

```
161 |             seen_keys.add(key)
162 |             results.append(art)
163 |         except ValueError:
164 |             pass
165 |
166 |     # 3. Поиск путей реестра
167 |     for match in cls._REGISTRY_REGEX.finditer(text):
168 |         reg = match.group(1)
169 |         art = ArtifactInput(
170 |             type=ArtifactType.REGISTRY_KEY,
171 |             registry_path=reg,
172 |         )
173 |         key = CanonicalKeyNormalizer.compute_canonical_key
174 |         (art)
175 |         if key not in seen_keys:
176 |             seen_keys.add(key)
177 |             results.append(art)
178 |
179 |     # 4. Если ничего специфичного не найдено, упаковываем
180 |     как симптом
181 |     if not results:
182 |         results.append(
183 |             ArtifactInput(
184 |                 type=ArtifactType.SYMPTOM,
185 |                 raw_query=text.strip()[:120],
186 |                 message=text,
187 |             )
188 |         )
189 |
190 |     return results
```

models.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Models
4 # =====
5 # Description:
6 #   Модели данных, перечисления и Pydantic-схемы для Progressive Knowledge Base WikiLLM.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.wikillm.models import ArtifactType
11 #
12 #       service = ArtifactType()
13 #
14 # File: models.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Модели данных, перечисления и Pydantic-схемы для Progressive Knowledge Base WikiLLM."""
24
25 from datetime import datetime, timezone
26 from enum import Enum
27 from typing import Any, Dict, List, Optional
28 from pydantic import BaseModel, Field
29
30
31 def utc_now_iso() -> str:
```

```

32     """Возвращает текущее время UTC в формате ISO 8601."""
33     return datetime.now(timezone.utc).isoformat()
34
35
36 class ArtifactType(str, Enum):
37     """Типы артефактов Windows и кодовой базы."""
38
39     WINDOWS_EVENT = "windows_event"
40     WINDOWS_ERROR = "windows_error"
41     REGISTRY_KEY = "registry_key"
42     PROCESS = "process"
43     SERVICE = "service"
44     DRIVER = "driver"
45     ETW_EVENT = "etw_event"
46     CODE_SYMBOL = "code_symbol"
47     INCIDENT = "incident"
48     SYMPTOM = "symptom"
49
50
51 class KnowledgeSource(str, Enum):
52     """Источники происхождения знаний в базе WikiLLM."""
53
54     OBSERVED = "observed"
55     DOCUMENTED = "documented"
56     INFERRED = "inferred"
57     LLM_GENERATED = "llm_generated"
58
59
60 class LookupLevel(str, Enum):
61     """Уровни многоступенчатого конвейера поиска."""
62
63     EXACT = "exact" # Level 1: точное совпадение
64     # по canonical_key
65     FINGERPRINT = "fingerprint" # Level 2: шаблонное совпаден
66     # ие
67     SEMANTIC = "semantic" # Level 3: FTS5 / семантическ
68     # ий поиск
69     GEMINI = "gemini" # Level 4: обращение к Gemini

```

LLM

```
67 UNKNOWN = "unknown"          # Не найдено
68
69
70 class ResolutionAction(BaseModel):
71     """Диагностическое действие или инструкция по устранению п
роблемы."""
72
73     title: str = Field(description="Заголовок действия")
74     description: str = Field(default="", description="Описание
шага")
75     command: Optional[str] = Field(default=None, description="
Команда CLI/PowerShell для выполнения")
76     risk_level: str = Field(default="safe", description="Урове
нь риска (safe/moderate/high)")
77     is_automated: bool = Field(default=False, description="Воз
можность автоматического запуска")
78
79
80 class Claim(BaseModel):
81     """Утверждение о сущности с фиксацией источника и верифика
ции."""
82
83     claim_id: Optional[str] = Field(default=None, description="
Уникальный ID утверждения")
84     statement: str = Field(description="Текст утверждения")
85     confidence: float = Field(default=1.0, ge=0.0, le=1.0, des
cription="Уверенность в утверждении")
86     source: KnowledgeSource = Field(default=KnowledgeSource.OB
SERVED, description="Источник факта")
87     verified: bool = Field(default=False, description="Провере
н ли факт документацией/наблюдением")
88     created_at: str = Field(default_factory=utc_now_iso, descr
iption="Время создания записи")
89
90
91 class Evidence(BaseModel):
92     """Свидетельство или наблюдаемый факт, подтверждающий знан
```

```

    ие. """
93
94     evidence_id: Optional[str] = Field(default=None, description="Идентификатор свидетельства")
95     description: str = Field(description="Описание наблюдаемого свидетельства")
96     telemetry_metric: Optional[str] = Field(default=None, description="Связанная телеметрическая метрика")
97     sample_value: Optional[str] = Field(default=None, description="Значение или вырезка лога")
98     observed_at: str = Field(default_factory=utc_now_iso, description="Время фиксации")
99
100
101 class ObservationRecord(BaseModel):
102     """Статистика наблюдений сущности на локальной машине."""
103
104     canonical_key: str = Field(description="Канонический ключ сущности")
105     total_occurrences: int = Field(default=1, description="Общее число наблюдений")
106     first_seen: str = Field(default_factory=utc_now_iso, description="Дата первого обнаружения")
107     last_seen: str = Field(default_factory=utc_now_iso, description="Дата последнего обнаружения")
108     co_occurrences: Dict[str, int] = Field(
109         default_factory=dict,
110         description="Совместные появления с другими canonical_key",
111     )
112
113
114 class DiagnosticKnowledge(BaseModel):
115     """Диагностические знания по ошибке или событию Windows."""
116
117     symptoms: List[str] = Field(default_factory=list, description="Наблюдаемые симптомы")

```

```

118     possible_causes: List[str] = Field(default_factory=list, d
description="Возможные причины")
119     diagnostic_actions: List[ResolutionAction] = Field(
120         default_factory=list,
121         description="Диагностические тесты и проверки",
122     )
123     remediation_steps: List[ResolutionAction] = Field(
124         default_factory=list,
125         description="Шаги по устранению проблемы",
126     )
127     related_components: List[str] = Field(
128         default_factory=list,
129         description="Связанные компоненты, службы или библиоте
ки",
130     )
131
132
133 class CodeKnowledge(BaseModel):
134     """Знания о программном коде и модулях."""
135
136     module_path: str = Field(default="", description="Путь к м
одулю в проекте")
137     symbol_name: str = Field(default="", description="Имя функ
ции, класса или метода")
138     symbol_type: str = Field(default="function", description="
Тип символа (class/function/method/api)")
139     docstring: str = Field(default="", description="Документац
ия символа")
140     parameters: List[str] = Field(default_factory=list, descri
ption="Параметры сигнатуры")
141     return_type: Optional[str] = Field(default=None, descripti
on="Возвращаемый тип")
142     dependencies: List[str] = Field(default_factory=list, desc
ription="Импортируемые зависимости")
143
144
145 class KnowledgeEntity(BaseModel):
146     """Главная модель сущности базы знаний WikiLLM."""

```

```
147
148     canonical_key: str = Field(description="Уникальный канонич
149     entity_type: ArtifactType = Field(description="Тип сущност
150     name: str = Field(description="Человекочитаемое имя или ид
151     summary: str = Field(description="Краткая сводка о назначе
152     category: str = Field(default="system", description="Катер
153     severity: str = Field(default="info", description="Критичн
154     confidence: float = Field(default=1.0, ge=0.0, le=1.0, des
155     provenance_source: KnowledgeSource = Field(
156         default=KnowledgeSource.OBSERVED,
157         description="Основной источник знания",
158     )
159     provenance_model: Optional[str] = Field(default=None, desc
160     diagnostic_info: Optional[DiagnosticKnowledge] = Field(
161         default=None,
162         description="Диагностические атрибуты",
163     )
164     code_info: Optional[CodeKnowledge] = Field(
165         default=None,
166         description="Атрибуты сущности кодовой базы",
167     )
168     claims: List[Claim] = Field(default_factory=list, descript
169     evidence: List[Evidence] = Field(default_factory=list, des
170     tags: List[str] = Field(default_factory=list, description=
171     fingerprint: Optional[str] = Field(default=None, descripti
172     created_at: str = Field(default_factory=utc_now_iso, descr
```

```

    172 |         ption="Время создания")
173 |         updated_at: str = Field(default_factory=utc_now_iso, descr
    174 |         ption="Время последнего обновления")
175 |
176 | class ArtifactInput(BaseModel):
177 |     """Входной артефакт для разрешения в WikiLLM."""
178 |
179 |     type: Optional[ArtifactType] = Field(default=None, descrip
    180 |     tion="Тип артефакта")
181 |     raw_query: Optional[str] = Field(default=None, description=
    182 |     "Сырая строка запроса/кода ошибки")
183 |     provider: Optional[str] = Field(default=None, description=
    184 |     "Windows Event Provider")
185 |     event_id: Optional[int] = Field(default=None, description=
    186 |     "Windows Event ID")
187 |     error_code: Optional[str] = Field(default=None, descriptio
    188 |     n="Код ошибки (0x8007..., 10016)")
189 |     process_name: Optional[str] = Field(default=None, descript
    190 |     ion="Имя процесса (svchost.exe)")
191 |     service_name: Optional[str] = Field(default=None, descript
    192 |     ion="Имя службы")
193 |     driver_name: Optional[str] = Field(default=None, descripti
    194 |     on="Имя драйвера")
195 |     registry_path: Optional[str] = Field(default=None, descrip
    196 |     tion="Путь в реестре Windows")
197 |     message: Optional[str] = Field(default=None, description="
    198 |     Текст сообщения или лога")
199 |     code_symbol: Optional[str] = Field(default=None, descripti
    200 |     on="Символ кода (модуль.класс)")
201 |     metadata: Dict[str, Any] = Field(default_factory=dict, des
    202 |     cription="Дополнительные метаданные")
203 |
204 | class ResolutionResult(BaseModel):
205 |     """Результат разрешения артефакта через 4-уровневый конвей
    206 |     ер WikiLLM."""
207 |

```

```
196 |     canonical_key: str = Field(description="Канонический ключ  
| артефакта")  
197 |     lookup_level: LookupLevel = Field(description="Уровень, на  
| котором найдено знание")  
198 |     cached: bool = Field(default=False, description="Было ли з  
| нание уже закэшировано в локальной БД")  
199 |     entity: Optional[KnowledgeEntity] = Field(default=None, de  
| scription="Разрешенная сущность базы знаний")  
200 |     confidence: float = Field(default=1.0, description="Итогов  
| ая уверенность")  
201 |     execution_time_ms: float = Field(default=0.0, description=  
| "Время обработки запроса в миллисекундах")  
202 |     message: str = Field(default="Успешно разрешено", descript  
| ion="Статусное сообщение")  
203 |
```

normalizer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Normalize
  r
4 # =====
5 # Description:
6 #   Нормализация входных артефактов Windows и кода в стабильны
  е канонические ключи
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.wikillm.normalizer import CanonicalKey
  Normalizer
11 #
12 #       service = CanonicalKeyNormalizer()
13 #
14 # File: normalizer.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Нормализация входных артефактов Windows и кода в стабильные
  канонические ключи"""
24
25 import hashlib
26 import re
27 from typing import Optional, Tuple
28 from .models import ArtifactInput, ArtifactType
29
```

```

30
31 class CanonicalKeyNormalizer:
32     """Генератор детерминированных канонических ключей и шабло
нов отпечатков."""
33
34     _HEX_ERROR_RE = re.compile(r"^(0x[0-9a-fA-F]{4,8}|-[0-9]{5
,10})$")
35     _GUID_RE = re.compile(r"\{?[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[
0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}\}?")
36     _HEX_ADDR_RE = re.compile(r"0x[0-9a-fA-F]{6,16}")
37     _TIMESTAMP_RE = re.compile(r"\d{4}-\d{2}-\d{2}[T\s]\d{2}:\
\d{2}:\d{2}(?:\.\d+)?(?:Z|[-+]\d{2}:\d{2})?")
38     _PATH_RE = re.compile(r"[a-zA-Z]:\\(?:[^\\"/*?\"<>|\r\n]+\\
\)*[^\\"/*?\"<>|\r\n]*")
39
40     @classmethod
41     def normalize_hex_code(cls, code: str) -> str:
42         """Нормализует шестнадцатеричный код ошибки к нижнему
регистру формата 0x...
43
44         Args:
45             code: Исходная строка кода ошибки (например, 0x800
70490, 80070490, -2147023728).
46
47         Returns:
48             Нормализованная строка шестнадцатеричного формата
(0x80070490).
49         """
50         raw = code.strip()
51         if raw.startswith("-"):
52             try:
53                 val = int(raw) & 0xFFFFFFFF
54                 return f"0x{val:08x}"
55             except ValueError:
56                 return raw.lower()
57
58         if raw.lower().startswith("0x"):
59             return raw.lower()

```

```
60 |
61 |         if len(raw) in (4, 8) and all(c in "0123456789abcdefAB
| CDEF" for c in raw):
62 |             return f"0x{raw.lower()}"
63 |
64 |         return raw.lower()
65 |
66 |     @classmethod
67 |     def normalize_registry_path(cls, path: str) -> str:
68 |         """Нормализует путь реестра к стандартным префиксам (H
| KLM, HKCU, HKCR).
69 |
70 |         Args:
71 |             path: Исходный путь реестра.
72 |
73 |         Returns:
74 |             Нормализованный путь реестра.
75 |         """
76 |         p = path.strip().replace("/", "\\")
77 |         replacements = [
78 |             ("HKEY_LOCAL_MACHINE\\", "HKLM\\"),
79 |             ("HKEY_CURRENT_USER\\", "HKCU\\"),
80 |             ("HKEY_CLASSES_ROOT\\", "HKCR\\"),
81 |             ("HKEY_USERS\\", "HKU\\"),
82 |             ("HKEY_CURRENT_CONFIG\\", "HKCC\\"),
83 |         ]
84 |         for src, dst in replacements:
85 |             if p.upper().startswith(src):
86 |                 p = dst + p[len(src):]
87 |                 break
88 |         return p
89 |
90 |     @classmethod
91 |     def compute_canonical_key(cls, artifact: ArtifactInput) ->
| str:
92 |         """Вычисляет стабильный канонический ключ для входного
| артефакта.
93 |
```

```
94 |         Args:
95 |             artifact: Входной объект артефакта.
96 |
97 |         Returns:
98 |             Строка канонического ключа (например win32:0x80070
100 | 490, windows_event:DistributedCOM:10016).
101 |         """
102 |         # 1. Если явно задан Event ID и Provider
103 |         if artifact.event_id is not None or (artifact.type ==
104 | ArtifactType.WINDOWS_EVENT and artifact.provider):
105 |             prov = (artifact.provider or "Generic").strip()
106 |             eid = artifact.event_id if artifact.event_id is no
107 | t None else 0
108 |             return f"windows_event:{prov}:{eid}"
109 |
110 |         # 2. Если задан код ошибки
111 |         if artifact.error_code:
112 |             norm_code = cls.normalize_hex_code(artifact.error_
113 | code)
114 |             prefix = "ntstatus" if norm_code.startswith("0xc")
115 | else "win32"
116 |             return f"{prefix}:{norm_code}"
117 |
118 |         # 3. Если задан путь реестра
119 |         if artifact.registry_path:
120 |             norm_reg = cls.normalize_registry_path(artifact.re
121 | gistry_path)
122 |             return f"registry:{norm_reg}"
123 |
124 |         # 4. Если задан процесс
125 |         if artifact.process_name:
126 |             proc = artifact.process_name.strip().lower()
127 |             return f"process:{proc}"
128 |
129 |         # 5. Если задана служба
130 |         if artifact.service_name:
131 |             srv = artifact.service_name.strip().lower()
132 |             return f"service:{srv}"
```

```

126
127     # 6. Если задан драйвер
128     if artifact.driver_name:
129         drv = artifact.driver_name.strip().lower()
130         return f"driver:{drv}"
131
132     # 7. Если задан символ кода
133     if artifact.code_symbol:
134         sym = artifact.code_symbol.strip()
135         return f"code:{sym}"
136
137     # 8. Анализ сырой строки запроса raw_query
138     if artifact.raw_query:
139         query = artifact.raw_query.strip()
140         # Проверка на код ошибки (0x8007..., 0xc000...)
141         if cls._HEX_ERROR_RE.match(query) or query.lower()
142         .startswith("0x"):
143             norm = cls.normalize_hex_code(query)
144             prefix = "ntstatus" if norm.startswith("0xc")
145             else "win32"
146             return f"{prefix}:{norm}"
147
148         # Проверка на шаблон "Event ID 10016" или "Distrib
149         utedCOM 10016"
150         ev_match = re.search(r"(?:event\s*id|eventid|event
151         )?\s*[:\s]?\s*([a-zA-Z0-9_\-]+)?\s*[:\s]?\s*(\d{1,6})", query,
152         re.IGNORECASE)
153         if ev_match and ev_match.group(2):
154             prov = ev_match.group(1) or "Generic"
155             return f"windows_event:{prov}:{ev_match.group(
156             2)}"
157
158         # Проверка на реестр
159         if any(query.upper().startswith(p) for p in ("HKLM
160         \\", "HKCU\\", "HKEY_")):
161             return f"registry:{cls.normalize_registry_path
162             (query)}"
163
164

```

```

156         # Проверка на процесс (*.exe)
157         if query.lower().endswith(".exe"):
158             return f"process:{query.lower()}"
159
160         # Проверка на драйвер (*.sys)
161         if query.lower().endswith(".sys"):
162             return f"driver:{query.lower()}"
163
164         # Общий симптом
165         slug = re.sub(r"[^a-zA-Z0-9_\-]+", "_", query.lower
166 r()).strip("_")
167         return f"symptom:{slug[:64]}"
168
169         # Резервный slug по сообщению
170         if artifact.message:
171             msg_snippet = re.sub(r"[^a-zA-Z0-9_\-]+", "_", art
172 ifact.message.lower()[:40]).strip("_")
173             return f"generic:{msg_snippet or 'unknown'}"
174
175         return "unknown:unspecified"
176
177     @classmethod
178     def sanitize_message_template(cls, message: str) -> str:
179         """Очищает текст сообщения от динамических данных (GUI
180 D, адреса, даты) для шаблонизации.
181
182     Args:
183         message: Исходное сырое сообщение из журнала или л
184 ога.
185
186     Returns:
187         Очищенный шаблон сообщения.
188     """
189     if not message:
190         return ""
191
192     s = cls._GUID_RE.sub("<GUID>", message)
193     s = cls._HEX_ADDR_RE.sub("<ADDR>", s)
194     s = cls._TIMESTAMP_RE.sub("<TIMESTAMP>", s)

```

```
190         s = cls._PATH_RE.sub("<PATH>", s)
191         s = re.sub(r"\b\d{4,}\b", "<NUM>", s)
192         return re.sub(r"\s+", " ", s).strip()
193
194     @classmethod
195     def compute_fingerprint(cls, artifact: ArtifactInput) -> str:
196         """Создает хеш-отпечаток (fingerprint) артефакта на основе структурных признаков.
197
198         Args:
199             artifact: Входной артефакт.
200
201         Returns:
202             Строка отпечатка или sha256-хеш структурного шаблона.
203
204         """
205         canonical_key = cls.compute_canonical_key(artifact)
206         msg_template = cls.sanitize_message_template(artifact.message or "")
207         prov = artifact.provider or ""
208         eid = str(artifact.event_id or "")
209
210         combined = f"{canonical_key}|{prov}|{eid}|{msg_template}"
211         return hashlib.sha256(combined.encode("utf-8")).hexdigest()[0:24]
```

base.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Resolvers -
  Base
4 # =====
5 # Description:
6 #   Базовый абстрактный интерфейс для уровней поиска и разреше
  ния артефактов.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.wikillm.resolvers.base import BaseReso
  lver
11 #
12 #       service = BaseResolver()
13 #
14 # File: base.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm.resolvers
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Базовый абстрактный интерфейс для уровней поиска и разреше
  ния артефактов."""
24
25 from abc import ABC, abstractmethod
26 from typing import Optional
27 from ..models import ArtifactInput, KnowledgeEntity, LookupLev
  el
28
```

```
29 |
30 | class BaseResolver(ABC):
31 |     """Абстрактный резолвер артефактов."""
32 |
33 |     def __init__(self, level: LookupLevel, name: str) -> None:
34 |         """Инициализирует базовый резолвер.
35 |
36 |         Args:
37 |             level: Уровень поиска (LookupLevel).
38 |             name: Человекочитаемое имя резолвера.
39 |         """
40 |         self.level = level
41 |         self.name = name
42 |
43 |     @abstractmethod
44 |     async def resolve(self, artifact: ArtifactInput) -> Option
45 | al[KnowledgeEntity]:
46 |         """Пытается разрешить артефакт на текущем уровне.
47 |
48 |         Args:
49 |             artifact: Входной артефакт.
50 |
51 |         Returns:
52 |             Сущность KnowledgeEntity или None, если знание не
53 |             найдено.
54 |         """
55 |         raise NotImplementedError
```

exact.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Resolvers -
  Exact
4 # =====
5 # Description:
6 #   Резолвер точного совпадения по каноническому ключу (canonical_key) в SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.wikillm.resolvers.exact import ExactResolver
11 #
12 #       service = ExactResolver()
13 #
14 # File: exact.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm.resolvers
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Резолвер точного совпадения по каноническому ключу (canonical_key) в SQLite."""
24
25 from typing import Optional
26 from ..models import ArtifactInput, KnowledgeEntity, LookupLevel
27 from ..normalizer import CanonicalKeyNormalizer
28 from ..storage import WikiStorage
```

```

29 from .base import BaseResolver
30
31
32 class ExactResolver(BaseResolver):
33     """Резолвер точного соответствия Level 1 (0(1) SQLite look
up)"""
34
35     def __init__(self, storage: WikiStorage) -> None:
36         """Инициализирует ExactResolver.
37
38         Args:
39             storage: Экземпляр хранилища WikiStorage.
40         """
41         super().__init__(level=LookupLevel.EXACT, name="ExactM
atchResolver")
42         self.storage = storage
43
44     async def resolve(self, artifact: ArtifactInput) -> Option
al[KnowledgeEntity]:
45         """Ищет запись в базе данных по canonical_key.
46
47         Args:
48             artifact: Входной артефакт.
49
50         Returns:
51             KnowledgeEntity при точном совпадении или None.
52         """
53         canonical_key = CanonicalKeyNormalizer.compute_canonic
al_key(artifact)
54         if not canonical_key or canonical_key == "unknown:unsp
ecified":
55             return None
56
57         return self.storage.get_entity(canonical_key)
58

```

fingerprint.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Resolvers -
  Fingerprint
4 # =====
5 # Description:
6 # Резолвер сопоставления по структурным шаблонам и отпечатка
  м (fingerprints).
7 #
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.wikillm.resolvers.fingerprint import F
  ingerprintResolver
11 #
12 # service = FingerprintResolver()
13 #
14 # File: fingerprint.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm.resolvers
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Резолвер сопоставления по структурным шаблонам и отпечаткам
  (fingerprints)."""
24
25 from typing import Optional
26 from ..models import ArtifactInput, KnowledgeEntity, LookupLev
  el
27 from ..normalizer import CanonicalKeyNormalizer
28 from ..storage import WikiStorage
```

```

29 from .base import BaseResolver
30
31
32 class FingerprintResolver(BaseResolver):
33     """Резолвер сопоставления по отпечаткам сообщений и провай
деров Level 2."""
34
35     def __init__(self, storage: WikiStorage) -> None:
36         """Инициализирует FingerprintResolver.
37
38         Args:
39             storage: Экземпляр хранилища WikiStorage.
40         """
41         super().__init__(level=LookupLevel.FINGERPRINT, name="
FingerprintTemplateResolver")
42         self.storage = storage
43
44     async def resolve(self, artifact: ArtifactInput) -> Option
al[KnowledgeEntity]:
45         """Ищет сущность по вычисленному структурному отпечатк
у.
46
47         Args:
48             artifact: Входной артефакт.
49
50         Returns:
51             KnowledgeEntity при шаблонном совпадении или None.
52         """
53         fingerprint = CanonicalKeyNormalizer.compute_fingerpri
nt(artifact)
54         if not fingerprint:
55             return None
56
57         return self.storage.find_by_fingerprint(fingerprint)
58

```

gemini.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Resolvers -
  Gemini
4 # =====
5 # Description:
6 #   Интеллектуальный генератор и структуризатор знаний на базе
  Google Gemini.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.wikillm.resolvers.gemini import Gemini
  KnowledgeResolver
11 #
12 #       service = GeminiKnowledgeResolver()
13 #
14 # File: gemini.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm.resolvers
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Интеллектуальный генератор и структуризатор знаний на базе
  Google Gemini."""
24
25 import json
26 import re
27 from typing import Any, Dict, List, Optional
28 from logger import logger
29 from ..models import (
```

```

30 |     ArtifactInput,
31 |     ArtifactType,
32 |     Claim,
33 |     DiagnosticKnowledge,
34 |     KnowledgeEntity,
35 |     KnowledgeSource,
36 |     LookupLevel,
37 |     ResolutionAction,
38 | )
39 | from ..normalizer import CanonicalKeyNormalizer
40 | from .base import BaseResolver
41 |
42 |
43 | _SYSTEM_PROMPT = """Ты — экспертный аналитик операционной сист
    | емы Windows и архитектор баз знаний.
44 | Твоя задача — проанализировать входящий артефакт Windows (Even
    | t ID, код ошибки HRESULT/Win32/NTSTATUS, имя процесса, ключ ре
    | естра, симптом) и вернуть СТРОГИЙ валидный JSON без markdown-о
    | берток (без ```json ... ```) со следующей структурой:
45 | {
46 |     "name": "Понятное наименование ошибки или артефакта",
47 |     "summary": "Четкое описание того, что означает данный артефа
    | кт",
48 |     "category": "system|security|storage|network|driver|code",
49 |     "severity": "info|warning|error|critical",
50 |     "confidence": 0.85,
51 |     "possible_causes": ["причина 1", "причина 2"],
52 |     "diagnostic_actions": [
53 |         {
54 |             "title": "Название проверки",
55 |             "description": "Что именно проверять",
56 |             "command": "powershell/cmd команда (если применимо)",
57 |             "risk_level": "safe",
58 |             "is_automated": true
59 |         }
60 |     ],
61 |     "remediation_steps": [
62 |         {

```

```

63         "title": "Шаг устранения",
64         "description": "Подробное действие",
65         "command": "powershell/cmd команда",
66         "risk_level": "safe",
67         "is_automated": false
68     }
69 ],
70     "related_components": ["DistributedCOM", "svchost.exe"],
71     "tags": ["dcom", "event-10016", "permissions"]
72 }
73 """
74
75
76 class GeminiKnowledgeResolver(BaseResolver):
77     """Резолвер синтеза знаний через Google Gemini с возвратом
78     валидированного JSON."""
79
80     def __init__(self, chat_model: Optional[Any] = None, model
81     _id: str = "gemini-flash-latest") -> None:
82         """Инициализирует GeminiKnowledgeResolver.
83
84         Args:
85             chat_model: Экземпляр адаптера Gemini Chat (опцион
86             ально).
87             model_id: Идентификатор модели Gemini по умолчанию
88             .
89
90         """
91         super().__init__(level=LookupLevel.GEMINI, name="Gemin
92         iKnowledgeResolver")
93         self.chat_model = chat_model
94         self.model_id = model_id
95
96     def _get_chat_client(self) -> Optional[Any]:
97         """Получает или лениво инициализирует адаптер Gemini."
98
99
100         if self.chat_model is not None:
101             return self.chat_model

```

```
95 |         try:
96 |             from src.ai.chat.gemini import GeminiChatBase
97 |             if GeminiChatBase.is_available():
98 |                 self.chat_model = GeminiChatBase(
99 |                     model_id=self.model_id,
100 |                     system_prompt=_SYSTEM_PROMPT,
101 |                 )
102 |                 return self.chat_model
103 |         except Exception as exc:
104 |             logger.debug(f"Не удалось инициализировать GeminiC
105 | hatBase: {exc}")
106 |
107 |         return None
108 |
109 |     def _build_prompt(self, artifact: ArtifactInput, canonical
110 | _key: str) -> str:
111 |         """Формирует структурированный запрос для Gemini."""
112 |         parts = [
113 |             f"Анализируй артефакт Windows:\nCanonical Key: {ca
114 | nonical_key}",
115 |             f"Тип: {artifact.type.value if artifact.type else
116 | 'unknown'}",
117 |         ]
118 |         if artifact.provider:
119 |             parts.append(f"Provider: {artifact.provider}")
120 |         if artifact.event_id is not None:
121 |             parts.append(f"Event ID: {artifact.event_id}")
122 |         if artifact.error_code:
123 |             parts.append(f"Код ошибки: {artifact.error_code}")
124 |         if artifact.process_name:
125 |             parts.append(f"Процесс: {artifact.process_name}")
126 |         if artifact.registry_path:
127 |             parts.append(f"Реестр: {artifact.registry_path}")
128 |         if artifact.message:
129 |             parts.append(f"Сообщение лога: {artifact.message}")
130 |
131 |         if artifact.raw_query:
132 |             parts.append(f"Сырой запрос: {artifact.raw_query}")
```

```

    )
128
129     parts.append("\nВерни СТРОГИЙ валидный JSON по заданно
й схеме.")
130     return "\n".join(parts)
131
132     def _parse_and_validate_json(self, response_text: str) ->
Optional[Dict[str, Any]]:
133         """Извлекает и валидирует JSON из текстового ответа LL
М."""
134         if not response_text:
135             return None
136
137         clean_text = response_text.strip()
138         # Очистка от markdown блоков ```json ... ```
139         if "```" in clean_text:
140             match = re.search(r"```(?:json)?\s*([\s\S]*?)\s*`
```", clean_text)
141 if match:
142 clean_text = match.group(1).strip()
143
144 try:
145 data = json.loads(clean_text)
146 if isinstance(data, dict):
147 return data
148 except Exception as exc:
149 logger.warning(f"Ошибка парсинга JSON ответа Gemin
i: {exc}. Текст: {response_text[:200]}")
150
151 return None
152
153 async def resolve(self, artifact: ArtifactInput) -> Option
al[KnowledgeEntity]:
154 """Обращается к Gemini, структурирует ответ и создает
объект KnowledgeEntity.
155
156 Args:
157 artifact: Входной артефакт.

```

```

158
159 Returns:
160 Сущность KnowledgeEntity или None при ошибке обрац
ения к LLM.
161 """
162 canonical_key = CanonicalKeyNormalizer.compute_canonic
al_key(artifact)
163 fingerprint = CanonicalKeyNormalizer.compute_fingerpri
nt(artifact)
164 client = self._get_chat_client()
165
166 if client is None:
167 logger.debug("Gemini клиент недоступен для разреше
ния артефакта.")
168 return None
169
170 prompt = self._build_prompt(artifact, canonical_key)
171 try:
172 raw_answer: Optional[str] = None
173 if hasattr(client, "ask"):
174 raw_answer = await client.ask(
175 prompt,
176 system_instruction=_SYSTEM_PROMPT,
177 temperature=0.1,
178)
179 elif hasattr(client, "send_message"):
180 raw_answer = await client.send_message(prompt)
181
182 if not raw_answer:
183 return None
184
185 payload = self._parse_and_validate_json(raw_answer
)
186
187 if not payload:
188 return None
189
190 # Построение структурированного объекта знаний
diag_actions = [

```

```

191 ResolutionAction(
192 title=a.get("title", ""),
193 description=a.get("description", ""),
194 command=a.get("command"),
195 risk_level=a.get("risk_level", "safe"),
196 is_automated=bool(a.get("is_automated", Fa
lse))),
197)
198 for a in payload.get("diagnostic_actions", [])
199 if isinstance(a, dict) and a.get("title")
200]
201
202 remed_steps = [
203 ResolutionAction(
204 title=s.get("title", ""),
205 description=s.get("description", ""),
206 command=s.get("command"),
207 risk_level=s.get("risk_level", "safe"),
208 is_automated=bool(s.get("is_automated", Fa
lse))),
209)
210 for s in payload.get("remediation_steps", [])
211 if isinstance(s, dict) and s.get("title")
212]
213
214 diag_info = DiagnosticKnowledge(
215 possible_causes=payload.get("possible_causes",
216 []),
217 diagnostic_actions=diag_actions,
218 remediation_steps=remed_steps,
219 related_components=payload.get("related_compon
220 ents", []),
221)
222
223 claims = [
224 Claim(
225 statement=f"Первопричина: {cause}",
226 confidence=float(payload.get("confidence",

```

```

 0.8)),
225 source=KnowledgeSource.LLM_GENERATED,
226 verified=False,
227)
228 for cause in payload.get("possible_causes", [])
229)
230
231 entity_type = artifact.type or ArtifactType.WINDOW
232 S_EVENT
233 entity = KnowledgeEntity(
234 canonical_key=canonical_key,
235 entity_type=entity_type,
236 name=payload.get("name", canonical_key),
237 summary=payload.get("summary", ""),
238 category=payload.get("category", "system"),
239 severity=payload.get("severity", "info"),
240 confidence=float(payload.get("confidence", 0.8
241 5)),
242 provenance_source=KnowledgeSource.LLM_GENERATE
243 D,
244 provenance_model=self.model_id,
245 fingerprint=fingerprint,
246 diagnostic_info=diag_info,
247 claims=claims,
248 tags=payload.get("tags", []),
249)
250
251 return entity
252
253 except Exception as exc:
254 logger.warning(f"Исключение при обращении к Gemini
255 в WikiLLM: {exc}")
256 return None
257

```

semantic.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Resolvers -
 Semantic
4 # =====
5 # Description:
6 # Резолвер семантического и полнотекстового поиска (FTS5). В
 выполняет поиск по смыслу,
7 #
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.wikillm.resolvers.semantic import Sema
 nticResolver
11 #
12 # service = SemanticResolver()
13 #
14 # File: semantic.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm.resolvers
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Резолвер семантического и полнотекстового поиска (FTS5). Вы
 полняет поиск по смыслу, """
24
25 from typing import Optional
26 from ..models import ArtifactInput, KnowledgeEntity, LookupLev
 el
27 from ..storage import WikiStorage
28 from .base import BaseResolver
```

```

29
30
31 class SemanticResolver(BaseResolver):
32 """Резолвер семантического сопоставления и полнотекстового
 поиска FTS5."""
33
34 def __init__(self, storage: WikiStorage) -> None:
35 """Инициализирует SemanticResolver.
36
37 Args:
38 storage: Экземпляр хранилища WikiStorage.
39 """
40 super().__init__(level=LookupLevel.SEMANTIC, name="SemanticFTSResolver")
41 self.storage = storage
42
43 async def resolve(self, artifact: ArtifactInput) -> Optional[KnowledgeEntity]:
44 """Выполняет полнотекстовый поиск по сообщению или описанию симптома.
45
46 Args:
47 artifact: Входной артефакт.
48
49 Returns:
50 KnowledgeEntity с наивысшим рангом релевантности или None.
51 """
52 search_query = artifact.message or artifact.raw_query
53 or ""
54 if not search_query or len(search_query.strip()) < 3:
55 return None
56
57 candidates = self.storage.search_fts(search_query, limit=1)
58
59 if candidates:
60 return candidates[0]

```

```
60 | return None
61 |
```

\_\_init\_\_.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Resolvers -
 Init
4 # =====
5 # Description:
6 # Экспорт классов резолверов многоступенчатого конвейера пои
 ска WikiLLM.
7 #
8 # Usage Examples:
9 #
10 #
11 # File: __init__.py
12 # Project: ai-breadboard
13 # Package: apps.windows.wikillm.resolvers
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # Updated: 2026-10-01 13:28:28
17 # =====
18
19 """Экспорт классов резолверов многоступенчатого конвейера пои
 ска WikiLLM."""
20
21 from .base import BaseResolver
22 from .exact import ExactResolver
23 from .fingerprint import FingerprintResolver
24 from .gemini import GeminiKnowledgeResolver
25 from .semantic import SemanticResolver
26
27 __all__ = [
28 "BaseResolver",
29 "ExactResolver",
30 "FingerprintResolver",
```

```
31 | "SemanticResolver",
32 | "GeminiKnowledgeResolver",
33 |]
34 |
```

router.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Router
4 # =====
5 # Description:
6 # REST API эндпоинты для разрешения артефактов, поиска по ба
 зе знаний WikiLLM,
7 #
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.wikillm.router import ResolveRequest
11 #
12 # service = ResolveRequest()
13 #
14 # File: router.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """REST API эндпоинты для разрешения артефактов, поиска по баз
 е знаний WikiLLM, """
24
25 from typing import Any, Dict, List, Optional
26 from fastapi import APIRouter, HTTPException, Query, Request
27 from pydantic import BaseModel, Field
28 from logger import logger
29 from .code_indexer import CodeKnowledgeIndexer
30 from .engine import WikiLLMEngine
31 from .extractor import ArtifactExtractor
```

```
32 from .models import (
33 ArtifactInput,
34 ArtifactType,
35 KnowledgeEntity,
36 LookupLevel,
37 ObservationRecord,
38 ResolutionResult,
39)
40
41
42 router = APIRouter(prefix="/api/windows/wikillm", tags=["windo
ws-wikillm"])
43 _engine_instance: Optional[WikiLLMEngine] = None
44
45
46 def get_engine() -> WikiLLMEngine:
47 """Возвращает или лениво инициализирует синглтон WikiLLMEn
gine."""
48 global _engine_instance
49 if _engine_instance is None:
50 _engine_instance = WikiLLMEngine()
51 return _engine_instance
52
53
54 def init_router(engine: Optional[WikiLLMEngine] = None) -> API
Router:
55 """Инициализирует роутер с внедрением экземпляра движка.
56
57 Args:
58 engine: Пользовательский экземпляр WikiLLMEngine.
59
60 Returns:
61 Сконфигурированный APIRouter.
62 """
63 global _engine_instance
64 if engine is not None:
65 _engine_instance = engine
66 return router
```

```

67 |
68 |
69 | class ResolveRequest(BaseModel):
70 | """Схема запроса на разрешение артефакта."""
71 |
72 | raw_query: Optional[str] = Field(default=None, description=
73 | "Строка ошибки или симптом")
74 | artifact: Optional[ArtifactInput] = Field(default=None, de
75 | scription="Типизированный объект артефакта")
76 | sync_gemini: bool = Field(default=True, description="Разре
77 | шать ли через Gemini синхронно")
78 | record_observation: bool = Field(default=True, description
79 | ="Фиксировать ли наблюдение")
80 |
81 |
82 | class CodeIngestRequest(BaseModel):
83 | """Схема запроса на индексацию кодовой базы."""
84 |
85 | target_path: str = Field(default="apps/windows", descripti
86 | on="Директория или файл для индексации")
87 |
88 |
89 | class ObserveRequest(BaseModel):
90 | """Схема фиксации наблюдения артефакта."""
91 |
92 | canonical_key: str = Field(description="Канонический ключ
93 | артефакта")
94 | co_occurring_keys: List[str] = Field(default_factory=list,
95 | description="Сопутствующие ключи")
96 |
97 |
98 | @router.post("/resolve", response_model=ResolutionResult)
99 | async def resolve_artifact(req: ResolveRequest) -> ResolutionR
100 | esult:
101 | """Разрешает артефакт по 4-уровневому конвейеру (Exact ->
102 | Fingerprint -> Semantic -> Gemini)."""
103 | engine = get_engine()

```

```

96 if req.artifact:
97 art = req.artifact
98 elif req.raw_query:
99 extracted = ArtifactExtractor.from_raw_text(req.raw_query)
100 art = extracted[0] if extracted else ArtifactInput(raw_query=req.raw_query)
101 else:
102 raise HTTPException(status_code=400, detail="Необходимо передать raw_query или artifact")
103
104 res = await engine.resolve(
105 art,
106 sync_gemini=req.sync_gemini,
107 record_observation=req.record_observation,
108)
109 return res
110
111
112 @router.get("/entities/{canonical_key:path}", response_model=KnowledgeEntity)
113 async def get_entity_by_key(canonical_key: str) -> KnowledgeEntity:
114 """Возвращает подробную информацию о сущности по её каноническому ключу."""
115 engine = get_engine()
116 entity = engine.storage.get_entity(canonical_key)
117 if not entity:
118 raise HTTPException(status_code=404, detail=f"Сущность не найдена: {canonical_key}")
119 return entity
120
121
122 @router.get("/entities", response_model=List[KnowledgeEntity])
123 async def list_entities(
124 limit: int = Query(default=50, ge=1, le=500),
125 offset: int = Query(default=0, ge=0),
126 type: Optional[str] = Query(default=None, description="Фил

```

```

 ытр по типу сущности"),
127) -> List[KnowledgeEntity]:
128 """Возвращает постраничный список сущностей базы знаний."""
129 engine = get_engine()
130 return engine.storage.list_entities(limit=limit, offset=offset, entity_type=type)
131
132
133 @router.get("/search", response_model=List[KnowledgeEntity])
134 async def search_knowledge(
135 q: str = Query(..., min_length=2, description="Поисковая строка"),
136 limit: int = Query(default=10, ge=1, le=50),
137) -> List[KnowledgeEntity]:
138 """Выполняет полнотекстовый поиск FTS5 по базе знаний Wiki LLM."""
139 engine = get_engine()
140 return engine.storage.search_fts(q, limit=limit)
141
142
143 @router.get("/observations/{canonical_key:path}", response_model=ObservationRecord)
144 async def get_observations(canonical_key: str) -> ObservationRecord:
145 """Возвращает статистику наблюдений артефакта на локальной машине."""
146 engine = get_engine()
147 obs = engine.storage.get_observation(canonical_key)
148 if not obs:
149 raise HTTPException(status_code=404, detail=f"Наблюдений для {canonical_key} не найдено")
150 return obs
151
152
153 @router.post("/observe", response_model=ObservationRecord)
154 async def record_observation_endpoint(req: ObserveRequest) -> ObservationRecord:

```

```
155 | """Вручную фиксирует наблюдение артефакта и совместные поя
| вления."""
156 | engine = get_engine()
157 | return engine.storage.record_observation(req.canonical_key
| , req.co_occurring_keys)
158 |
159 |
160 | @router.post("/code/ingest")
161 | async def ingest_code(req: CodeIngestRequest) -> Dict[str, Any
|]:
162 | """Запускает индексацию AST структуры исходного кода кодов
| ой базы."""
163 | engine = get_engine()
164 | indexer = CodeKnowledgeIndexer(engine.storage)
165 | count = indexer.index_directory(req.target_path)
166 | return {
167 | "status": "success",
168 | "target_path": req.target_path,
169 | "indexed_symbols_count": count,
170 | }
171 |
172 |
173 | @router.get("/stats")
174 | async def get_system_stats() -> Dict[str, Any]:
175 | """Возвращает метрики эффективности, статистику кэш-хитов
| и состояние хранилища."""
176 | engine = get_engine()
177 | return engine.get_metrics()
178 |
```

storage.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Storage
4 # =====
5 # Description:
6 # Хранилище базы знаний WikiLLM на SQLite с поддержкой полно
 текстового поиска FTS5,
7 #
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.wikillm.storage import WikiStorage
11 #
12 # service = WikiStorage()
13 #
14 # File: storage.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Хранилище базы знаний WikiLLM на SQLite с поддержкой полнот
 екстового поиска FTS5, """
24
25 import json
26 import sqlite3
27 import uuid
28 from contextlib import contextmanager
29 from pathlib import Path
30 from typing import Any, Dict, Generator, List, Optional
31 from logger import logger
```

```

32 from .models import (
33 ArtifactType,
34 Claim,
35 CodeKnowledge,
36 DiagnosticKnowledge,
37 Evidence,
38 KnowledgeEntity,
39 KnowledgeSource,
40 ObservationRecord,
41 ResolutionAction,
42 utc_now_iso,
43)
44
45
46 class WikiStorage:
47 """SQLite-хранилище сущностей, фактов, наблюдений и полнотекстового индекса."""
48
49 def __init__(self, db_path: str | Path = "data/windows_wikillm/knowledge.db") -> None:
50 """Инициализирует подключение к SQLite базе данных.
51
52 Args:
53 db_path: Путь к файлу базы данных или ':memory:'.
54 """
55 self.db_path = Path(db_path) if str(db_path) != ":memory:" else ":memory:"
56 self._mem_conn: Optional[sqlite3.Connection] = None
57 if self.db_path == ":memory:":
58 self._mem_conn = sqlite3.connect(":memory:", check_same_thread=False)
59 self._mem_conn.row_factory = sqlite3.Row
60 self._mem_conn.execute("PRAGMA foreign_keys = ON")
61 elif isinstance(self.db_path, Path):
62 self.db_path.parent.mkdir(parents=True, exist_ok=True)
63
64 self._init_db()

```

```

65 | @contextmanager
66 | def _connect(self) -> Generator[sqlite3.Connection, None,
None]:
67 | """Контекстный менеджер соединения с базой данных SQLite."""
68 | if self._mem_conn is not None:
69 | try:
70 | yield self._mem_conn
71 | self._mem_conn.commit()
72 | except Exception:
73 | self._mem_conn.rollback()
74 | raise
75 | else:
76 | conn = sqlite3.connect(str(self.db_path), check_same_thread=False)
77 | conn.row_factory = sqlite3.Row
78 | conn.execute("PRAGMA foreign_keys = ON")
79 | try:
80 | yield conn
81 | conn.commit()
82 | finally:
83 | conn.close()
84 |
85 | def _init_db(self) -> None:
86 | """Создает таблицы базы данных и виртуальную таблицу F
TS5 при их отсутствии."""
87 | with self._connect() as conn:
88 | conn.executescript(
89 | """
90 | CREATE TABLE IF NOT EXISTS entities (
91 | canonical_key TEXT PRIMARY KEY,
92 | entity_type TEXT NOT NULL,
93 | name TEXT NOT NULL,
94 | summary TEXT NOT NULL,
95 | category TEXT NOT NULL DEFAULT 'system',
96 | severity TEXT NOT NULL DEFAULT 'info',
97 | confidence REAL NOT NULL DEFAULT 1.0,
98 | provenance_source TEXT NOT NULL DEFAULT 'o

```

```

 bserved',
99 provenance_model TEXT,
100 fingerprint TEXT,
101 diagnostic_json TEXT,
102 code_json TEXT,
103 tags_json TEXT,
104 created_at TEXT NOT NULL,
105 updated_at TEXT NOT NULL
106);
107
108 CREATE TABLE IF NOT EXISTS observations (
109 canonical_key TEXT PRIMARY KEY,
110 total_occurrences INTEGER NOT NULL DEFAULT
111 1,
112 first_seen TEXT NOT NULL,
113 last_seen TEXT NOT NULL,
114 co_occurrences_json TEXT NOT NULL DEFAULT
115 '{}')
116);
117
118 CREATE TABLE IF NOT EXISTS claims (
119 claim_id TEXT PRIMARY KEY,
120 canonical_key TEXT NOT NULL REFERENCES entities(canonical_key) ON DELETE CASCADE,
121 statement TEXT NOT NULL,
122 confidence REAL NOT NULL DEFAULT 1.0,
123 source TEXT NOT NULL DEFAULT 'observed',
124 verified INTEGER NOT NULL DEFAULT 0,
125 created_at TEXT NOT NULL
126);
127
128 CREATE TABLE IF NOT EXISTS evidence (
129 evidence_id TEXT PRIMARY KEY,
130 canonical_key TEXT NOT NULL REFERENCES entities(canonical_key) ON DELETE CASCADE,
131 description TEXT NOT NULL,
132 telemetry_metric TEXT,
133 sample_value TEXT,

```

```

132 observed_at TEXT NOT NULL
133);
134
135 CREATE TABLE IF NOT EXISTS relationships (
136 source_key TEXT NOT NULL,
137 target_key TEXT NOT NULL,
138 relationship_type TEXT NOT NULL,
139 confidence REAL NOT NULL DEFAULT 1.0,
140 co_occurrences INTEGER NOT NULL DEFAULT 1,
141 PRIMARY KEY(source_key, target_key, relationship_type)
142);
143
144 CREATE TABLE IF NOT EXISTS resolutions (
145 id INTEGER PRIMARY KEY AUTOINCREMENT,
146 canonical_key TEXT NOT NULL REFERENCES entities(canonical_key) ON DELETE CASCADE,
147 title TEXT NOT NULL,
148 description TEXT NOT NULL,
149 command TEXT,
150 risk_level TEXT NOT NULL DEFAULT 'safe',
151 is_automated INTEGER NOT NULL DEFAULT 0
152);
153
154 CREATE INDEX IF NOT EXISTS idx_entities_fingerprint ON entities(fingerprint);
155 CREATE INDEX IF NOT EXISTS idx_entities_category ON entities(category);
156 CREATE INDEX IF NOT EXISTS idx_entities_type ON entities(entity_type);
157 CREATE INDEX IF NOT EXISTS idx_claims_key ON claims(canonical_key);
158 CREATE INDEX IF NOT EXISTS idx_evidence_key ON evidence(canonical_key);
159 CREATE INDEX IF NOT EXISTS idx_rel_source ON relationships(source_key);
160
161 CREATE VIRTUAL TABLE IF NOT EXISTS source_fts

```

```

162 USING fts5(
163 canonical_key UNINDEXED,
164 name,
165 summary,
166 tags,
167 details
168);
169 """
170
171 def save_entity(self, entity: KnowledgeEntity) -> None:
172 """Сохраняет или обновляет сущность базы знаний со все
173 ми связями и FTS-индексом.
174
175 Args:
176 entity: Экземпляр KnowledgeEntity для сохранения.
177 """
178 now = utc_now_iso()
179 diag_json = entity.diagnostic_info.model_dump_json() i
180 f entity.diagnostic_info else None
181 code_json = entity.code_info.model_dump_json() if enti
182 ty.code_info else None
183 tags_json = json.dumps(entity.tags, ensure_ascii=False
184)
185
186 with self._connect() as conn:
187 # 1. Вставка или обновление главной записи entitie
188 s
189 conn.execute(
190 """
191 INSERT INTO entities (
192 canonical_key, entity_type, name, summary,
193 category, severity,
194 confidence, provenance_source, provenance_
195 model, fingerprint,
196 diagnostic_json, code_json, tags_json, cre
197 ated_at, updated_at
198) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,

```

```

?, ?, ?)
191 ON CONFLICT(canonical_key) DO UPDATE SET
192 entity_type = excluded.entity_type,
193 name = excluded.name,
194 summary = excluded.summary,
195 category = excluded.category,
196 severity = excluded.severity,
197 confidence = excluded.confidence,
198 provenance_source = excluded.provenance_source,
199 provenance_model = excluded.provenance_model,
200 fingerprint = excluded.fingerprint,
201 diagnostic_json = excluded.diagnostic_json
202 ,
203 code_json = excluded.code_json,
204 tags_json = excluded.tags_json,
205 updated_at = excluded.updated_at
206 """,
207 (
208 entity.canonical_key,
209 entity.entity_type.value,
210 entity.name,
211 entity.summary,
212 entity.category,
213 entity.severity,
214 entity.confidence,
215 entity.provenance_source.value,
216 entity.provenance_model,
217 entity.fingerprint,
218 diag_json,
219 code_json,
220 tags_json,
221 entity.created_at or now,
222 now,
223),
224)

```

```

225 # 2. Обновление claims
226 conn.execute("DELETE FROM claims WHERE canonical_k
ey = ?", (entity.canonical_key,))
227 for claim in entity.claims:
228 cid = claim.claim_id or str(uuid.uuid4())
229 conn.execute(
230 """
231 INSERT INTO claims (claim_id, canonical_ke
y, statement, confidence, source, verified, created_at)
232 VALUES (?, ?, ?, ?, ?, ?, ?)
233 """,
234 (
235 cid,
236 entity.canonical_key,
237 claim.statement,
238 claim.confidence,
239 claim.source.value,
240 1 if claim.verified else 0,
241 claim.created_at or now,
242),
243)
244
245 # 3. Обновление evidence
246 conn.execute("DELETE FROM evidence WHERE canonical
_key = ?", (entity.canonical_key,))
247 for ev in entity.evidence:
248 eid = ev.evidence_id or str(uuid.uuid4())
249 conn.execute(
250 """
251 INSERT INTO evidence (evidence_id, canonic
al_key, description, telemetry_metric, sample_value, observed_
at)
252 VALUES (?, ?, ?, ?, ?, ?)
253 """,
254 (
255 eid,
256 entity.canonical_key,
257 ev.description,

```

```

258 ev.telemetry_metric,
259 ev.sample_value,
260 ev.observed_at or now,
261),
262)
263
264 # 4. Обновление resolutions
265 conn.execute("DELETE FROM resolutions WHERE canonical_key = ?", (entity.canonical_key,))
266 if entity.diagnostic_info:
267 actions = entity.diagnostic_info.remediation_steps + entity.diagnostic_info.diagnostic_actions
268 for act in actions:
269 conn.execute(
270 """
271 INSERT INTO resolutions (canonical_key
272 , title, description, command, risk_level, is_automated)
273 VALUES (?, ?, ?, ?, ?, ?)
274 """,
275 (
276 entity.canonical_key,
277 act.title,
278 act.description,
279 act.command,
280 act.risk_level,
281 1 if act.is_automated else 0,
282),
283)
284
285 # 5. Обновление FTS5 индекса
286 conn.execute("DELETE FROM source_fts WHERE canonical_key = ?", (entity.canonical_key,))
287 details_text = f"{diag_json or ''} {code_json or ''}"
288 conn.execute(
289 """
290 INSERT INTO source_fts (canonical_key, name, summary, tags, details)

```

```

290 VALUES (?, ?, ?, ?, ?)
291 """,
292 (
293 entity.canonical_key,
294 entity.name,
295 entity.summary,
296 " ".join(entity.tags),
297 details_text,
298),
299)
300
301 def get_entity(self, canonical_key: str) -> Optional[KnowledgeEntity]:
302 """Загружает сущность по каноническому ключу со всеми
303 сопутствующими данными.
304
305 Args:
306 canonical_key: Канонический ключ сущности.
307
308 Returns:
309 KnowledgeEntity или None, если сущность не найдена
310 .
311
312 """
313 with self._connect() as conn:
314 row = conn.execute(
315 "SELECT * FROM entities WHERE canonical_key = ?", (canonical_key,)
316).fetchone()
317 if not row:
318 return None
319
320 claims_rows = conn.execute(
321 "SELECT * FROM claims WHERE canonical_key = ?", (canonical_key,)
322).fetchall()
323 evidence_rows = conn.execute(
324 "SELECT * FROM evidence WHERE canonical_key = ?", (canonical_key,)

```

```

322).fetchall()
323
324 claims = [
325 Claim(
326 claim_id=r["claim_id"],
327 statement=r["statement"],
328 confidence=r["confidence"],
329 source=KnowledgeSource(r["source"]),
330 verified=bool(r["verified"]),
331 created_at=r["created_at"],
332)
333 for r in claims_rows
334]
335
336 evidence = [
337 Evidence(
338 evidence_id=r["evidence_id"],
339 description=r["description"],
340 telemetry_metric=r["telemetry_metric"],
341 sample_value=r["sample_value"],
342 observed_at=r["observed_at"],
343)
344 for r in evidence_rows
345]
346
347 diag_info = None
348 if row["diagnostic_json"]:
349 try:
350 diag_info = DiagnosticKnowledge.model_validate_json(
351 row["diagnostic_json"])
352 except Exception:
353 pass
354
355 code_info = None
356 if row["code_json"]:
357 try:
358 code_info = CodeKnowledge.model_validate_json(
359 row["code_json"])

```

```

358 except Exception:
359 pass
360
361 tags = []
362 if row["tags_json"]:
363 try:
364 tags = json.loads(row["tags_json"])
365 except Exception:
366 pass
367
368 return KnowledgeEntity(
369 canonical_key=row["canonical_key"],
370 entity_type=ArtifactType(row["entity_type"]),
371 name=row["name"],
372 summary=row["summary"],
373 category=row["category"],
374 severity=row["severity"],
375 confidence=row["confidence"],
376 provenance_source=KnowledgeSource(row["provena
nce_source"]),
377 provenance_model=row["provenance_model"],
378 fingerprint=row["fingerprint"],
379 diagnostic_info=diag_info,
380 code_info=code_info,
381 claims=claims,
382 evidence=evidence,
383 tags=tags,
384 created_at=row["created_at"],
385 updated_at=row["updated_at"],
386)
387
388 def find_by_fingerprint(self, fingerprint: str) -> Optiona
l[KnowledgeEntity]:
389 """Ищет сущность по шаблону отпечатка (fingerprint).
390
391 Args:
392 fingerprint: Строка отпечатка.
393

```

```

394 | Returns:
395 | KnowledgeEntity или None.
396 | """
397 | if not fingerprint:
398 | return None
399 | with self._connect() as conn:
400 | row = conn.execute(
401 | "SELECT canonical_key FROM entities WHERE fing
erprint = ? LIMIT 1", (fingerprint,)
402 |).fetchone()
403 | if row:
404 | return self.get_entity(row["canonical_key"])
405 | return None
406 |
407 | def search_fts(self, query: str, limit: int = 10) -> List[
KnowledgeEntity]:
408 | """Выполняет полнотекстовый поиск по FTS5 индексу.
409 |
410 | Args:
411 | query: Поисковый запрос.
412 | limit: Максимальное количество результатов.
413 |
414 | Returns:
415 | Список найденных KnowledgeEntity.
416 | """
417 | cleaned_query = "".join(c if c.isalnum() or c.isspace(
) else " " for c in query).strip()
418 | if not cleaned_query:
419 | return []
420 |
421 | fts_query = " OR ".join(f'"{token}"' for token in cle
aned_query.split() if token)
422 | if not fts_query:
423 | return []
424 |
425 | results: List[KnowledgeEntity] = []
426 | with self._connect() as conn:
427 | try:

```

```

428 rows = conn.execute(
429 """
430 SELECT canonical_key, rank FROM source_fts
431 WHERE source_fts MATCH ?
432 ORDER BY rank
433 LIMIT ?
434 """,
435 (fts_query, limit),
436).fetchall()
437 except sqlite3.OperationalError:
438 # В случае синтаксической ошибки FTS используе
439 m LIKE fallback
440 rows = conn.execute(
441 """
442 SELECT canonical_key FROM entities
443 WHERE name LIKE ? OR summary LIKE ?
444 LIMIT ?
445 """,
446 (f"%{query}%", f"%{query}%", limit),
447).fetchall()
448 for r in rows:
449 ent = self.get_entity(r["canonical_key"])
450 if ent:
451 results.append(ent)
452
453 return results
454
455 def record_observation(
456 self, canonical_key: str, co_occurring_keys: Optional[
457 List[str]] = None
458) -> ObservationRecord:
459 """Фиксирует локальное наблюдение артефакта и обновляе
460 т счетчики совместных появлений.
461
462 Args:
463 canonical_key: Канонический ключ артефакта.
464 co_occurring_keys: Список одновременно наблюдавших

```

ся других артефактов.

```
463
464 Returns:
465 Обновленная запись ObservationRecord.
466 """
467 now = utc_now_iso()
468 co_keys = [k for k in (co_occurring_keys or []) if k != canonical_key]
469
470 with self._connect() as conn:
471 row = conn.execute(
472 "SELECT * FROM observations WHERE canonical_key = ?", (canonical_key,)
473).fetchone()
474
475 if row:
476 total = row["total_occurrences"] + 1
477 co_map: Dict[str, int] = {}
478 try:
479 co_map = json.loads(row["co_occurrences_json"])
480 except Exception:
481 pass
482
483 for k in co_keys:
484 co_map[k] = co_map.get(k, 0) + 1
485
486 conn.execute(
487 """
488 UPDATE observations
489 SET total_occurrences = ?, last_seen = ?,
490 co_occurrences_json = ?
491 WHERE canonical_key = ?
492 """,
493 (total, now, json.dumps(co_map, ensure_ascii=False), canonical_key),
494)
495 rec = ObservationRecord(
```

```

495 canonical_key=canonical_key,
496 total_occurrences=total,
497 first_seen=row["first_seen"],
498 last_seen=now,
499 co_occurrences=co_map,
500)
501 else:
502 co_map = {k: 1 for k in co_keys}
503 conn.execute(
504 """
505 INSERT INTO observations (canonical_key, t
otal_occurrences, first_seen, last_seen, co_occurrences_json)
506 VALUES (?, 1, ?, ?, ?)
507 """,
508 (canonical_key, now, now, json.dumps(co_ma
p, ensure_ascii=False)),
509)
510 rec = ObservationRecord(
511 canonical_key=canonical_key,
512 total_occurrences=1,
513 first_seen=now,
514 last_seen=now,
515 co_occurrences=co_map,
516)
517
518 # Обновление графа отношений
519 for target_k, count in co_map.items():
520 conn.execute(
521 """
522 INSERT INTO relationships (source_key, tar
get_key, relationship_type, confidence, co_occurrences)
523 VALUES (?, ?, 'co_occurs_with', 0.8, ?)
524 ON CONFLICT(source_key, target_key, relati
onship_type) DO UPDATE SET
525 co_occurrences = excluded.co_occurrenc
es,
526 confidence = MIN(1.0, 0.5 + (excluded.
co_occurrences * 0.05))

```

```

527 """
528 (canonical_key, target_k, count),
529)
530
531 return rec
532
533 def get_observation(self, canonical_key: str) -> Optional[
534 ObservationRecord]:
535 """Возвращает историю наблюдений сущности.
536
537 Args:
538 canonical_key: Канонический ключ.
539
540 Returns:
541 ObservationRecord или None.
542 """
543 with self._connect() as conn:
544 row = conn.execute(
545 "SELECT * FROM observations WHERE canonical_ke
546 y = ?", (canonical_key,)
547).fetchone()
548 if not row:
549 return None
550 co_map = {}
551 try:
552 co_map = json.loads(row["co_occurrences_json"])
553)
554 except Exception:
555 pass
556 return ObservationRecord(
557 canonical_key=row["canonical_key"],
558 total_occurrences=row["total_occurrences"],
559 first_seen=row["first_seen"],
560 last_seen=row["last_seen"],
561 co_occurrences=co_map,
562)
563
564 def get_stats(self) -> Dict[str, Any]:

```

```

562 | """Возвращает общую статистику базы знаний WikiLLM.
563 |
564 | Returns:
565 | Словарь с метриками (число сущностей, наблюдений,
| категорий).
566 | """
567 | with self._connect() as conn:
568 | total_entities = conn.execute("SELECT COUNT(*) FROM
| M entities").fetchone()[0]
569 | total_obs = conn.execute("SELECT COUNT(*), SUM(tot
| al_occurrences) FROM observations").fetchone()
570 | total_claims = conn.execute("SELECT COUNT(*) FROM
| claims").fetchone()[0]
571 | cat_rows = conn.execute(
572 | "SELECT category, COUNT(*) as c FROM entities
| GROUP BY category"
573 |).fetchall()
574 | type_rows = conn.execute(
575 | "SELECT entity_type, COUNT(*) as c FROM entiti
| es GROUP BY entity_type"
576 |).fetchall()
577 |
578 | return {
579 | "total_entities": total_entities,
580 | "unique_observed_artifacts": total_obs[0] if total
| _obs else 0,
581 | "total_observation_events": (total_obs[1] or 0) if
| total_obs else 0,
582 | "total_claims": total_claims,
583 | "categories": {r["category"]: r["c"] for r in cat_
| rows},
584 | "types": {r["entity_type"]: r["c"] for r in type_r
| ows},
585 | }
586 |
587 | def list_entities(
588 | self, limit: int = 50, offset: int = 0, entity_type: O
| ptional[str] = None

```

```

589 |) -> List[KnowledgeEntity]:
590 | """Возвращает постраничный список сущностей базы знани
 | й.
591 |
592 | Args:
593 | limit: Количество записей.
594 | offset: Смещение.
595 | entity_type: Фильтр по типу сущности (опционально)
 | .
596 |
597 | Returns:
598 | Список сущностей KnowledgeEntity.
599 | """
600 | results: List[KnowledgeEntity] = []
601 | with self._connect() as conn:
602 | if entity_type:
603 | rows = conn.execute(
604 | "SELECT canonical_key FROM entities WHERE
 | entity_type = ? ORDER BY updated_at DESC LIMIT ? OFFSET ?",
605 | (entity_type, limit, offset),
606 |).fetchall()
607 | else:
608 | rows = conn.execute(
609 | "SELECT canonical_key FROM entities ORDER
 | BY updated_at DESC LIMIT ? OFFSET ?",
610 | (limit, offset),
611 |).fetchall()
612 |
613 | for r in rows:
614 | ent = self.get_entity(r["canonical_key"])
615 | if ent:
616 | results.append(ent)
617 |
618 | return results
619 |

```

## telemetry\_bridge.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Telemetry
 Bridge
4 # =====
5 # Description:
6 # Мост интеграции между подсистемой телеметрии Windows (IncidentDetector,
7 #
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.wikillm.telemetry_bridge import TelemetryWikiBridge
11 #
12 # service = TelemetryWikiBridge()
13 #
14 # File: telemetry_bridge.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Мост интеграции между подсистемой телеметрии Windows (IncidentDetector, """
24
25 from typing import Any, Dict, List, Optional
26 from logger import logger
27 from .engine import WikiLLMEngine
28 from .extractor import ArtifactExtractor
29 from .models import ArtifactInput, ResolutionResult
```

```

30 |
31 |
32 | class TelemetryWikiBridge:
33 | """Мост между телеметрией Windows и базой знаний WikiLLM."
34 | """
35 | def __init__(self, engine: WikiLLMEngine) -> None:
36 | """Инициализирует мост телеметрии.
37 |
38 | Args:
39 | engine: Экземпляр движка WikiLLMEngine.
40 | """
41 | self.engine = engine
42 |
43 | async def enrich_incident(
44 | self, incident_data: Dict[str, Any], sync_gemini: bool
45 | = False
46 |) -> Dict[str, Any]:
47 | """Обогащает инцидент телеметрии структурированными зн
48 | аниями из WikiLLM.
49 |
50 | Args:
51 | incident_data: Словарь с описанием инцидента.
52 | sync_gemini: Флаг синхронного вызова Gemini при от
53 | сутствии данных в кэше.
54 |
55 | Returns:
56 | Обогащенный словарь инцидента с секцией 'wiki_know
57 | ledge'.
58 | """
59 | artifacts = ArtifactExtractor.from_incident(incident_d
60 | ata)
61 | enriched_knowledge: List[Dict[str, Any]] = []
62 | all_causes: List[str] = []
63 | all_actions: List[Dict[str, Any]] = []
64 |
65 | co_occurring_keys = [
66 | self.engine.exact_resolver.storage.db_path

```

```

62 for _ in [1]
63] # placeholder if needed
64
65 extracted_keys = []
66 for art in artifacts:
67 res: ResolutionResult = await self.engine.resolve(
68 art,
69 sync_gemini=sync_gemini,
70 record_observation=True,
71)
72 extracted_keys.append(res.canonical_key)
73 if res.entity:
74 item_data = {
75 "canonical_key": res.canonical_key,
76 "name": res.entity.name,
77 "summary": res.entity.summary,
78 "severity": res.entity.severity,
79 "confidence": res.confidence,
80 "lookup_level": res.lookup_level.value,
81 }
82 if res.entity.diagnostic_info:
83 item_data["possible_causes"] = res.entity.
diagnostic_info.possible_causes
84 item_data["remediation_steps"] = [
85 step.model_dump() for step in res.entit
ty.diagnostic_info.remediation_steps
86]
87 all_causes.extend(res.entity.diagnostic_in
fo.possible_causes)
88 all_actions.extend([s.model_dump() for s i
n res.entity.diagnostic_info.remediation_steps])
89
90 enriched_knowledge.append(item_data)
91
92 # Добавляем историю наблюдений и обогащенную сводку
93 incident_copy = dict(incident_data)
94 incident_copy["wiki_enrichment"] = {
95 "resolved_entities": enriched_knowledge,

```

```

96 "suggested_causes": list(dict.fromkeys(all_causes)
97),
98 "recommended_actions": all_actions,
99 "total_artifacts_analyzed": len(artifacts),
100 }
101 return incident_copy
102
103 async def ingest_event_stream(
104 self, events: List[Dict[str, Any]], sync_gemini: bool
105 = False
106) -> int:
107 """Принимает поток событий и отправляет их на нормализ
108 ацию и разрешение.
109
110 Args:
111 events: Список словарей событий Windows.
112 sync_gemini: Флаг синхронного обращения к Gemini (
113 False для массовой телеметрии).
114
115 Returns:
116 Число успешно обработанных событий.
117 """
118 count = 0
119 for ev in events:
120 art = ArtifactExtractor.from_event_dict(ev)
121 await self.engine.resolve(art, sync_gemini=sync_ge

```

test\_cli.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Tests - Test Cli
4 # =====
5 # Description:
6 # Тесты для CLI интерфейса и configurатора WikiLLM.
7 #
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.wikillm.tests.test_cli import test_config_defaults
11 #
12 # res = test_config_defaults()
13 #
14 # File: test_cli.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm.tests
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Тесты для CLI интерфейса и configurатора WikiLLM."""
24
25 import argparse
26 from unittest.mock import AsyncMock, patch
27 import pytest
28 from apps.windows.wikillm.cli import _async_cli_main
29 from apps.windows.wikillm.config import WikiLLMConfig, load_config
30
```

```
31
32 def test_config_defaults() -> None:
33 """Проверка загрузки конфигурации по умолчанию."""
34 cfg = load_config(None)
35 assert cfg.database_path == "data/windows_wikillm/knowledg
 e.db"
36 assert cfg.min_confidence_threshold == 0.70
37 assert cfg.enable_semantic_search is True
38
39
40 @pytest.mark.asyncio
41 async def test_cli_search_and_stats(capsys: pytest.CaptureFixt
 ure) -> None:
42 """Проверка команд search и stats в CLI."""
43 args_stats = argparse.Namespace(command="stats")
44 await _async_cli_main(args_stats)
45 captured = capsys.readouterr()
46 assert "lookup_stats" in captured.out
47
48 args_search = argparse.Namespace(command="search", query="
 test", limit=5)
49 await _async_cli_main(args_search)
50 captured = capsys.readouterr()
51 assert "Найдено результатов" in captured.out
52
53
54 @pytest.mark.asyncio
55 async def test_cli_index_code(capsys: pytest.CaptureFixture) -
 > None:
56 """Проверка команды index-code в CLI."""
57 args = argparse.Namespace(command="index-code", path="apps
 /windows/wikillm")
58 await _async_cli_main(args)
59 captured = capsys.readouterr()
60 assert "Успешно проиндексировано" in captured.out
61
```

test\_code\_indexer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Tests - Test Code Indexer
4 # =====
5 # Description:
6 # Тесты для AST индексатора структуры Python кода (Code Knowledge).
7 #
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.wikillm.tests.test_code_indexer import
11 # test_index_python_file
12 # res = test_index_python_file()
13 #
14 # File: test_code_indexer.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm.tests
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Тесты для AST индексатора структуры Python кода (Code Knowledge)."""
24
25 import tempfile
26 from pathlib import Path
27 import pytest
28 from apps.windows.wikillm.code_indexer import CodeKnowledgeIndexer
```

```

29 from apps.windows.wikillm.storage import WikiStorage
30
31
32 def test_index_python_file() -> None:
33 """Проверка извлечения классов, функций и docstrings через
34 AST."""
35 storage = WikiStorage(":memory:")
36 indexer = CodeKnowledgeIndexer(storage)
37
38 code_snippet = '''"""Тестовый модуль для проверки индексат
39 opa."""
40
41 import os
42 from typing import Optional
43
44 class SampleDiagnosticTool:
45 """Класс для системной диагностики."""
46
47 def run_check(self, target: str) -> bool:
48 """Выполняет проверку целевого компонента."""
49 return True
50
51 def standalone_helper(param1: int) -> str:
52 """Вспомогательная функция."""
53 return str(param1)
54
55 with tempfile.TemporaryDirectory() as tmp_dir:
56 tmp_file = Path(tmp_dir) / "sample_mod.py"
57 tmp_file.write_text(code_snippet, encoding="utf-8")
58
59 entities = indexer.index_file(tmp_file, base_dir=tmp_d
60 ir)
61
62 assert len(entities) == 3
63
64 keys = [e.canonical_key for e in entities]
65 assert any("SampleDiagnosticTool" in k for k in keys)
66 assert any("run_check" in k for k in keys)
67 assert any("standalone_helper" in k for k in keys)
68

```

```
64 | # Проверяем, что сущности сохранены в хранилище и дост
 | упны по FTS
65 | search_res = storage.search_fts("системной диагностики
 | ")
66 | assert len(search_res) >= 1
67 |
```

test\_engine.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Tests - Test Engine
4 # =====
5 # Description:
6 # Тесты для движка WikiLLMEngine, сквозного цикла самообучения (первый вызов Gemini ->
7 #
8 # Usage Examples:
9 # Python API:
10 # import apps.windows.wikillm.tests.test_engine as test_engine
11 #
12 # File: test_engine.py
13 # Project: ai-breadboard
14 # Package: apps.windows.wikillm.tests
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-01 13:28:28
18 # =====
19
20 from __future__ import annotations
21 """Тесты для движка WikiLLMEngine, сквозного цикла самообучения (первый вызов Gemini ->"""
22
23 import asyncio
24 import json
25 from unittest.mock import AsyncMock
26 import pytest
27 from apps.windows.wikillm.config import WikiLLMConfig
28 from apps.windows.wikillm.engine import WikiLLMEngine
29 from apps.windows.wikillm.models import ArtifactInput, Artific
```

```

tType, LookupLevel
30 from apps.windows.wikillm.storage import WikiStorage
31 from apps.windows.wikillm.telemetry_bridge import TelemetryWik
 iBridge
32
33
34 @pytest.mark.asyncio
35 async def test_progressive_learning_cycle() -> None:
36 """Проверка полного цикла прогрессивного обучения:
37
38 1. Первый запрос: UNKNOWN в кэше -> обращение к Gemini ->
 запись в SQLite.
39 2. Второй запрос: KNOWN в SQLite -> Level 1 Exact Match, 0
 обращений к Gemini, мгновенно.
40 """
41 storage = WikiStorage(":memory:")
42 mock_chat = AsyncMock()
43 mock_gemini_response = {
44 "name": "ERROR_ACCESS_DENIED",
45 "summary": "Отказано в доступе к системному объекту ил
 и ресурсу",
46 "category": "security",
47 "severity": "error",
48 "confidence": 0.92,
49 "possible_causes": ["Отсутствие прав администратора",
 "Блокировка UAC"],
50 "diagnostic_actions": [],
51 "remediation_steps": [
52 {
53 "title": "Перезапуск от администратора",
54 "description": "Запустить терминал от имени ад
 министра",
55 "command": "powershell Start-Process -Verb run
 As",
56 "risk_level": "safe",
57 "is_automated": False,
58 }
59],

```

```

60 "related_components": ["Security", "UAC"],
61 "tags": ["access_denied", "0x80070005"],
62 }
63 mock_chat.ask.return_value = json.dumps(mock_gemini_response)
64
65 cfg = WikiLLMConfig(database_path=":memory:")
66 engine = WikiLLMEngine(config=cfg, storage=storage, chat_model=mock_chat)
67
68 art = ArtifactInput(type=ArtifactType.WINDOWS_ERROR, error_code="0x80070005")
69
70 # --- ПЕРВЫЙ ЗАПРОС (Обучение) ---
71 res1 = await engine.resolve(art, sync_gemini=True)
72 assert res1.lookup_level == LookupLevel.GEMINI
73 assert res1.cached is False
74 assert res1.entity is not None
75 assert res1.entity.name == "ERROR_ACCESS_DENIED"
76 assert mock_chat.ask.call_count == 1
77
78 # --- ВТОРОЙ ЗАПРОС (Мгновенный повтор) ---
79 res2 = await engine.resolve(art, sync_gemini=True)
80 assert res2.lookup_level == LookupLevel.EXACT
81 assert res2.cached is True
82 assert res2.entity is not None
83 assert res2.entity.name == "ERROR_ACCESS_DENIED"
84 # Счётчик вызовов Gemini НЕ увеличился!
85 assert mock_chat.ask.call_count == 1
86
87 # --- ТРЕТИЙ ЗАПРОС (Проверка счетчика наблюдений) ---
88 res3 = await engine.resolve(art, sync_gemini=True)
89 assert res3.lookup_level == LookupLevel.EXACT
90 assert mock_chat.ask.call_count == 1
91
92 # Проверка метрик кэша
93 metrics = engine.get_metrics()
94 assert metrics["lookup_stats"]["total_requests"] == 3

```

```

95 | assert metrics["lookup_stats"]["l1_exact_hits"] == 2
96 | assert metrics["lookup_stats"]["l4_gemini_resolutions"] ==
 | 1
97 | assert metrics["cache_hit_rate_pct"] > 60.0
98 |
99 |
100 | @pytest.mark.asyncio
101 | async def test_telemetry_bridge_incident_enrichment() -> None:
102 | """Проверка обогащения инцидента телеметрии через Telemetr
 | yWikiBridge."""
103 | storage = WikiStorage(":memory:")
104 | mock_chat = AsyncMock()
105 | mock_gemini_response = {
106 | "name": "Disk Anomaly",
107 | "summary": "Аномальная запись на диск процессом",
108 | "category": "storage",
109 | "severity": "warning",
110 | "confidence": 0.85,
111 | "possible_causes": ["Интенсивное резервное копирование
 | "],
112 | "diagnostic_actions": [],
113 | "remediation_steps": [
114 | {
115 | "title": "Проверить диск",
116 | "description": "Проверить SMART статус",
117 | "command": "wmic diskdrive get status",
118 | "risk_level": "safe",
119 | }
120 |],
121 | "related_components": ["Storage"],
122 | "tags": ["disk"],
123 | }
124 | mock_chat.ask.return_value = json.dumps(mock_gemini_respon
 | se)
125 |
126 | cfg = WikiLLMConfig(database_path=":memory:")
127 | engine = WikiLLMEngine(config=cfg, storage=storage, chat_m
 | odel=mock_chat)

```

```
128 | bridge = TelemetryWikiBridge(engine)
129 |
130 | raw_incident = {
131 | "id": "inc-123",
132 | "category": "storage",
133 | "trigger_type": "disk_burst",
134 | "description": "Disk write 500MB/s",
135 | "suspects": [{"name": "system.exe"}],
136 | }
137 |
138 | enriched = await bridge.enrich_incident(raw_incident, sync
|_gemini=True)
139 | assert "wiki_enrichment" in enriched
140 | assert len(enriched["wiki_enrichment"]["resolved_entities"
|]) > 0
141 | assert "Интенсивное резервное копирование" in enriched["wi
| ki_enrichment"]["suggested_causes"]
142 |
```

test\_extractor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Tests - Test Extractor
4 # =====
5 # Description:
6 # Тесты для извлечения артефактов из логов, словарей событий
7 # и инцидентов.
8 #
9 # Usage Examples:
10 # Python API:
11 # from apps.windows.wikillm.tests.test_extractor import test_extract_from_event_dict
12 # res = test_extract_from_event_dict()
13 #
14 # File: test_extractor.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm.tests
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Тесты для извлечения артефактов из логов, словарей событий
24 и инцидентов."""
25
26 import pytest
27 from apps.windows.wikillm.extractor import ArtifactExtractor
28 from apps.windows.wikillm.models import ArtifactType
29
```

```

30 def test_extract_from_event_dict() -> None:
31 """Проверка извлечения артефакта из словаря Windows Event.
 """
32 ev = {
33 "Provider": "DistributedCOM",
34 "EventID": 10016,
35 "ProcessName": "svchost.exe",
36 "Message": "The application-specific permission settin
 gs do not grant Local Activation permission.",
37 }
38 art = ArtifactExtractor.from_event_dict(ev)
39 assert art.type == ArtifactType.WINDOWS_EVENT
40 assert art.provider == "DistributedCOM"
41 assert art.event_id == 10016
42 assert art.process_name == "svchost.exe"
43
44
45 def test_extract_from_incident() -> None:
46 """Проверка извлечения артефактов из объекта инцидента."""
47 inc = {
48 "category": "storage",
49 "trigger_type": "disk_burst",
50 "description": "High write rate detected",
51 "suspects": [{"name": "system.exe", "pid": 4}, {"name"
 : "backup.exe"}],
52 "error_code": "0x80070005",
53 }
54 artifacts = ArtifactExtractor.from_incident(inc)
55 assert len(artifacts) >= 3
56
57 types = [a.type for a in artifacts]
58 assert ArtifactType.INCIDENT in types
59 assert ArtifactType.PROCESS in types
60 assert ArtifactType.WINDOWS_ERROR in types
61
62
63 def test_extract_from_raw_text() -> None:
64 """Проверка извлечения артефактов из свободного текста."""

```

```
65 | raw = "Failed to open registry HKLM\\Software\\Policies wi
 | th error 0x80070490 in Event ID 7034"
66 | artifacts = ArtifactExtractor.from_raw_text(raw)
67 |
68 | types = [a.type for a in artifacts]
69 | assert ArtifactType.WINDOWS_ERROR in types
70 | assert ArtifactType.WINDOWS_EVENT in types
71 | assert ArtifactType.REGISTRY_KEY in types
72 |
```

test\_normalizer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Tests - Test Normalizer
4 # =====
5 # Description:
6 # Тесты для нормализатора канонических ключей и шаблонов артефактов.
7 #
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.wikillm.tests.test_normalizer import test_normalize_hex_code
11 #
12 # res = test_normalize_hex_code()
13 #
14 # File: test_normalizer.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm.tests
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Тесты для нормализатора канонических ключей и шаблонов артефактов."""
24
25 import pytest
26 from apps.windows.wikillm.models import ArtifactInput, ArtifactType
27 from apps.windows.wikillm.normalizer import CanonicalKeyNormalizer
```

```

28
29
30 def test_normalize_hex_code() -> None:
31 """Проверка нормализации шестнадцатеричных кодов ошибок."""
32 assert CanonicalKeyNormalizer.normalize_hex_code("0x800704
33 90") == "0x80070490"
34 assert CanonicalKeyNormalizer.normalize_hex_code("80070490
35 ") == "0x80070490"
36 assert CanonicalKeyNormalizer.normalize_hex_code("0X800700
37 05") == "0x80070005"
38 assert CanonicalKeyNormalizer.normalize_hex_code("-2147024
39 891") == "0x80070005"
40
41
42 def test_normalize_registry_path() -> None:
43 """Проверка нормализации путей реестра."""
44 assert (
45 CanonicalKeyNormalizer.normalize_registry_path(
46 "HKEY_LOCAL_MACHINE\\Software\\Microsoft\\Windows\\
47 \\CurrentVersion\\Run"
48)
49 == "HKLM\\Software\\Microsoft\\Windows\\CurrentVersion
50 \\Run"
51)
52 assert (
53 CanonicalKeyNormalizer.normalize_registry_path("HKEY_C
54 UURRENT_USER/Software/App")
55 == "HKCU\\Software\\App"
56)
57
58
59 def test_canonical_key_generation() -> None:
60 """Проверка генерации канонических ключей для разных типов
61 артефактов."""
62 # 1. Windows Event
63 art_ev = ArtifactInput(
64 type=ArtifactType.WINDOWS_EVENT,

```

```

57 provider="DistributedCOM",
58 event_id=10016,
59)
60 assert CanonicalKeyNormalizer.compute_canonical_key(art_ev
61) == "windows_event:DistributedCOM:10016"
62
63 # 2. Win32 Error
64 art_err = ArtifactInput(
65 type=ArtifactType.WINDOWS_ERROR,
66 error_code="0x80070490",
67)
68 assert CanonicalKeyNormalizer.compute_canonical_key(art_er
69 r) == "win32:0x80070490"
70
71 # 3. NTSTATUS Error
72 art_nt = ArtifactInput(
73 type=ArtifactType.WINDOWS_ERROR,
74 error_code="0xc0000005",
75)
76 assert CanonicalKeyNormalizer.compute_canonical_key(art_nt
77) == "ntstatus:0xc0000005"
78
79 # 4. Process
80 art_proc = ArtifactInput(
81 type=ArtifactType.PROCESS,
82 process_name="Svchost.EXE",
83)
84 assert CanonicalKeyNormalizer.compute_canonical_key(art_pr
85 oc) == "process:svchost.exe"
86
87 # 5. Service
88 art_srv = ArtifactInput(
89 type=ArtifactType.SERVICE,
90 service_name="wuauserv",
91)
92 assert CanonicalKeyNormalizer.compute_canonical_key(art_sr
93 v) == "service:wuauserv"
94

```

```
90 | # 6. Raw query error code
91 | art_raw = ArtifactInput(raw_query="0x80070005")
92 | assert CanonicalKeyNormalizer.compute_canonical_key(art_raw) == "win32:0x80070005"
93 |
94 |
95 | def test_fingerprint_generation() -> None:
96 | """Проверка очистки шаблона и генерации стабильного отпечатка."""
97 | msg1 = "The application svchost.exe at address 0x7FFE1234AB failed with GUID {12345678-1234-1234-1234-123456789abc} on 2026-10-01T07:00:00Z"
98 | msg2 = "The application svchost.exe at address 0x7FFE9876CD failed with GUID {87654321-4321-4321-4321-cba987654321} on 2026-10-02T10:15:30Z"
99 |
100 | art1 = ArtifactInput(provider="AppCrash", event_id=1000, message=msg1)
101 | art2 = ArtifactInput(provider="AppCrash", event_id=1000, message=msg2)
102 |
103 | fp1 = CanonicalKeyNormalizer.compute_fingerprint(art1)
104 | fp2 = CanonicalKeyNormalizer.compute_fingerprint(art2)
105 |
106 | assert fp1 == fp2, "Отпечатки с динамическими GUID и адресами должны совпадать!"
107 |
```

## test\_resolvers.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Tests - Test Resolvers
4 # =====
5 # Description:
6 # Тесты для 4 уровней резолверов (Exact, Fingerprint, Semantic, Gemini).
7 #
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.wikillm.tests.test_resolvers import storage
11 #
12 # res = storage()
13 #
14 # File: test_resolvers.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm.tests
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Тесты для 4 уровней резолверов (Exact, Fingerprint, Semantic, Gemini)."""
24
25 import json
26 from unittest.mock import AsyncMock
27 import pytest
28 from apps.windows.wikillm.models import (
29 ArtifactInput,
```

```

30 ArtifactType,
31 KnowledgeEntity,
32 LookupLevel,
33)
34 from apps.windows.wikillm.normalizer import CanonicalKeyNormal
 izer
35 from apps.windows.wikillm.resolvers.exact import ExactResolver
36 from apps.windows.wikillm.resolvers.fingerprint import FingerprintResolver
37 from apps.windows.wikillm.resolvers.gemini import GeminiKnowledgeResolver
38 from apps.windows.wikillm.resolvers.semantic import SemanticResolver
39 from apps.windows.wikillm.storage import WikiStorage
40
41
42 @pytest.fixture
43 def storage() -> WikiStorage:
44 """Изолированное хранилище."""
45 return WikiStorage(":memory:")
46
47
48 @pytest.mark.asyncio
49 async def test_exact_resolver_hit_and_miss(storage: WikiStorage) -> None:
50 """Проверка Level 1 ExactResolver."""
51 resolver = ExactResolver(storage)
52
53 art = ArtifactInput(
54 type=ArtifactType.WINDOWS_EVENT,
55 provider="DistributedCOM",
56 event_id=10016,
57)
58 # 1. До сохранения - промах
59 assert await resolver.resolve(art) is None
60
61 # 2. Сохраняем сущность
62 storage.save_entity(

```

```

63 KnowledgeEntity(
64 canonical_key="windows_event:DistributedCOM:10016"
65 ,
66 entity_type=ArtifactType.WINDOWS_EVENT,
67 name="DCOM 10016",
68 summary="DCOM Warning",
69)
70)
71 # 3. После сохранения - точный хит
72 res = await resolver.resolve(art)
73 assert res is not None
74 assert res.canonical_key == "windows_event:DistributedCOM:
10016"
75
76
77 @pytest.mark.asyncio
78 async def test_fingerprint_resolver(storage: WikiStorage) -> N
one:
79 """Проверка Level 2 FingerprintResolver."""
80 resolver = FingerprintResolver(storage)
81
82 template_art = ArtifactInput(
83 provider="TestProv",
84 event_id=5000,
85 message="Failure in 0x7FFE1234 with GUID {12345678-123
4-1234-1234-123456789abc}",
86)
87 fp = CanonicalKeyNormalizer.compute_fingerprint(template_a
rt)
88
89 storage.save_entity(
90 KnowledgeEntity(
91 canonical_key="test:5000",
92 entity_type=ArtifactType.WINDOWS_EVENT,
93 name="Test Event",
94 summary="Test summary",
95 fingerprint=fp,

```

```

96)
97)
98
99 # Приходит запрос с ДРУГИМ адресом и GUID, но тем же шабло
НОМ
100 incoming_art = ArtifactInput(
101 provider="TestProv",
102 event_id=5000,
103 message="Failure in 0x7FFE9999 with GUID {87654321-432
1-4321-4321-cba987654321}",
104)
105 res = await resolver.resolve(incoming_art)
106 assert res is not None
107 assert res.canonical_key == "test:5000"
108
109
110 @pytest.mark.asyncio
111 async def test_semantic_resolver(storage: WikiStorage) -> None
:
112 """Проверка Level 3 SemanticResolver."""
113 resolver = SemanticResolver(storage)
114
115 storage.save_entity(
116 KnowledgeEntity(
117 canonical_key="win32:0x80070005",
118 entity_type=ArtifactType.WINDOWS_ERROR,
119 name="Access Denied",
120 summary="Отказано в доступе из-за отсутствия приви
легий администратора",
121)
122)
123
124 art = ArtifactInput(raw_query="отсутствия привилегий админ
истратора")
125 res = await resolver.resolve(art)
126 assert res is not None
127 assert res.canonical_key == "win32:0x80070005"
128

```

```
129 |
130 | @pytest.mark.asyncio
131 | async def test_gemini_resolver_with_mock() -> None:
132 | """Проверка Level 4 GeminiKnowledgeResolver со структуриро
 ванным JSON ответом."""
133 | mock_chat = AsyncMock()
134 | mock_payload = {
135 | "name": "ERROR_NOT_FOUND",
136 | "summary": "Элемент не найден в хранилище компонентов
 Windows",
137 | "category": "system",
138 | "severity": "error",
139 | "confidence": 0.88,
140 | "possible_causes": ["Повреждение хранилища WinSxS", "Н
 еверный путь обновления"],
141 | "diagnostic_actions": [
142 | {
143 | "title": "Проверка DISM",
144 | "description": "Запуск сканирования хранилища"
145 | ,
146 | "command": "DISM.exe /Online /Cleanup-image /S
 canHealth",
147 | "risk_level": "safe",
148 | "is_automated": True,
149 | }
150 |],
151 | "remediation_steps": [
152 | {
153 | "title": "Восстановление DISM",
154 | "description": "Восстановление поврежденных ко
 мпонентов",
155 | "command": "DISM.exe /Online /Cleanup-image /R
 estoreHealth",
156 | "risk_level": "safe",
157 | "is_automated": True,
158 | }
159 |],
160 | "related_components": ["DISM", "WinSxS", "CBS"],
```

```
160 "tags": ["dism", "winsxs", "0x80070490"],
161 }
162 mock_chat.ask.return_value = json.dumps(mock_payload)
163
164 resolver = GeminiKnowledgeResolver(chat_model=mock_chat)
165 art = ArtifactInput(type=ArtifactType.WINDOWS_ERROR, error
166 _code="0x80070490")
167
168 res = await resolver.resolve(art)
169 assert res is not None
170 assert res.canonical_key == "win32:0x80070490"
171 assert res.name == "ERROR_NOT_FOUND"
172 assert res.diagnostic_info is not None
173 assert len(res.diagnostic_info.remediation_steps) == 1
174 assert "DISM" in res.diagnostic_info.remediation_steps[0].
175 command
176 assert res.confidence == 0.88
```

test\_router.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Tests - Test Router
4 # =====
5 # Description:
6 # Тесты для REST API эндпоинтов WikiLLM с использованием TestClient.
7 #
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.wikillm.tests.test_router import client
11 #
12 # res = client()
13 #
14 # File: test_router.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm.tests
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Тесты для REST API эндпоинтов WikiLLM с использованием TestClient."""
24
25 import pytest
26 from fastapi import FastAPI
27 from fastapi.testclient import TestClient
28 from apps.windows.wikillm.config import WikiLLMConfig
29 from apps.windows.wikillm.engine import WikiLLMEngine
```

```

30 from apps.windows.wikillm.models import ArtifactType, KnowledgeEntity
31 from apps.windows.wikillm.router import init_router
32 from apps.windows.wikillm.storage import WikiStorage
33
34
35 @pytest.fixture
36 def client() -> TestClient:
37 """Фикстура TestClient с изолированным движком WikiLLM."""
38 app = FastAPI()
39 storage = WikiStorage(":memory:")
40 cfg = WikiLLMConfig(database_path=:memory:)
41 engine = WikiLLMEngine(config=cfg, storage=storage)
42
43 # Предзаполняем тестовой сущностью
44 storage.save_entity(
45 KnowledgeEntity(
46 canonical_key="win32:0x80070005",
47 entity_type=ArtifactType.WINDOWS_ERROR,
48 name="Access Denied",
49 summary="Access is denied test error",
50 tags=["access", "security"],
51)
52)
53
54 router = init_router(engine)
55 app.include_router(router)
56 return TestClient(app)
57
58
59 def test_resolve_endpoint(client: TestClient) -> None:
60 """Проверка POST /api/windows/wikillm/resolve."""
61 response = client.post(
62 "/api/windows/wikillm/resolve",
63 json={"raw_query": "0x80070005", "sync_gemini": False}
64),
65 assert response.status_code == 200

```

```

66 data = response.json()
67 assert data["canonical_key"] == "win32:0x80070005"
68 assert data["cached"] is True
69 assert data["lookup_level"] == "exact"
70 assert data["entity"]["name"] == "Access Denied"
71
72
73 def test_get_entity_endpoint(client: TestClient) -> None:
74 """Проверка GET /api/windows/wikillm/entities/{key}."""
75 response = client.get("/api/windows/wikillm/entities/win32
:0x80070005")
76 assert response.status_code == 200
77 data = response.json()
78 assert data["canonical_key"] == "win32:0x80070005"
79 assert data["name"] == "Access Denied"
80
81
82 def test_list_entities_endpoint(client: TestClient) -> None:
83 """Проверка GET /api/windows/wikillm/entities."""
84 response = client.get("/api/windows/wikillm/entities")
85 assert response.status_code == 200
86 data = response.json()
87 assert len(data) >= 1
88
89
90 def test_search_endpoint(client: TestClient) -> None:
91 """Проверка GET /api/windows/wikillm/search."""
92 response = client.get("/api/windows/wikillm/search", param
s={"q": "Access denied"})
93 assert response.status_code == 200
94 data = response.json()
95 assert len(data) >= 1
96 assert data[0]["canonical_key"] == "win32:0x80070005"
97
98
99 def test_stats_endpoint(client: TestClient) -> None:
100 """Проверка GET /api/windows/wikillm/stats."""
101 response = client.get("/api/windows/wikillm/stats")

```

```
102 | assert response.status_code == 200
103 | data = response.json()
104 | assert "lookup_stats" in data
105 | assert "storage_stats" in data
106 |
```

## test\_storage.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Tests - Test Storage
4 # =====
5 # Description:
6 # Тесты для SQLite хранилища, FTS5 поиска, наблюдений и связей
7 # ей сущностей.
8 #
9 # Usage Examples:
10 # Python API:
11 # from apps.windows.wikillm.tests.test_storage import memory_storage
12 # res = memory_storage()
13 #
14 # File: test_storage.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm.tests
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Тесты для SQLite хранилища, FTS5 поиска, наблюдений и связей
24 ей сущностей."""
25
26 import pytest
27 from apps.windows.wikillm.models import (
28 ArtifactType,
29 Claim,
```

```

30 Evidence,
31 KnowledgeEntity,
32 KnowledgeSource,
33 ResolutionAction,
34)
35 from apps.windows.wikillm.storage import WikiStorage
36
37
38 @pytest.fixture
39 def memory_storage() -> WikiStorage:
40 """Фикстура изолированного хранилища в памяти."""
41 return WikiStorage(":memory:")
42
43
44 def test_save_and_get_entity(memory_storage: WikiStorage) -> N
one:
45 """Проверка сохранения и загрузки полной сущности базы зна
ний."""
46 diag = DiagnosticKnowledge(
47 possible_causes=["Недостаточно прав DCOM"],
48 remediation_steps=[
49 ResolutionAction(
50 title="Настроить dcomcnfg",
51 description="Открыть Component Services и наст
роить права",
52 command="dcomcnfg",
53 risk_level="safe",
54)
55],
56 related_components=["DistributedCOM", "svchost.exe"],
57)
58
59 entity = KnowledgeEntity(
60 canonical_key="windows_event:DistributedCOM:10016",
61 entity_type=ArtifactType.WINDOWS_EVENT,
62 name="DCOM Permission Warning",
63 summary="Параметры разрешений для конкретного приложен
ия не дают разрешения Локально Активация",

```

```

64 category="system",
65 severity="warning",
66 confidence=0.9,
67 provenance_source=KnowledgeSource.OBSERVED,
68 fingerprint="dcom_10016_fp",
69 diagnostic_info=diag,
70 claims=[
71 Claim(statement="Обычно безвредно для работы систе
мы", confidence=0.95, verified=True)
72],
73 evidence=[
74 Evidence(description="Наблюдалось при запуске DCOM
сервера", sample_value="AppID {1234}")
75],
76 tags=["dcom", "event10016"],
77)
78
79 memory_storage.save_entity(entity)
80
81 loaded = memory_storage.get_entity("windows_event:Distribu
tedCOM:10016")
82 assert loaded is not None
83 assert loaded.name == "DCOM Permission Warning"
84 assert loaded.severity == "warning"
85 assert loaded.diagnostic_info is not None
86 assert len(loaded.diagnostic_info.remediation_steps) == 1
87 assert loaded.diagnostic_info.remediation_steps[0].command
== "dcomcnfg"
88 assert len(loaded.claims) == 1
89 assert loaded.claims[0].verified is True
90
91
92 def test_find_by_fingerprint(memory_storage: WikiStorage) -> N
one:
93 """Проверка поиска по отпечатку."""
94 entity = KnowledgeEntity(
95 canonical_key="win32:0x80070490",
96 entity_type=ArtifactType.WINDOWS_ERROR,

```

```

97 name="ERROR_NOT_FOUND",
98 summary="Элемент не найден",
99 fingerprint="fp_0x80070490_test",
100)
101 memory_storage.save_entity(entity)
102
103 found = memory_storage.find_by_fingerprint("fp_0x80070490_
test")
104 assert found is not None
105 assert found.canonical_key == "win32:0x80070490"
106
107
108 def test_fts5_search(memory_storage: WikiStorage) -> None:
109 """Проверка полнотекстового поиска FTS5."""
110 entity1 = KnowledgeEntity(
111 canonical_key="service:wuauserv",
112 entity_type=ArtifactType.SERVICE,
113 name="Windows Update Service",
114 summary="Служба автоматического обновления операционно
й системы Windows",
115 tags=["updates", "windows_update"],
116)
117 entity2 = KnowledgeEntity(
118 canonical_key="service:spooler",
119 entity_type=ArtifactType.SERVICE,
120 name="Print Spooler",
121 summary="Диспетчер очереди печати принтеров",
122 tags=["print", "spooler"],
123)
124 memory_storage.save_entity(entity1)
125 memory_storage.save_entity(entity2)
126
127 results = memory_storage.search_fts("автоматического обнов
ления")
128 assert len(results) == 1
129 assert results[0].canonical_key == "service:wuauserv"
130
131

```

```
132 def test_record_observations_and_co_occurrences(memory_storage
 : WikiStorage) -> None:
133 """Проверка счетчиков наблюдений и графа совместных появле
 ний."""
134 obs1 = memory_storage.record_observation("process:svchost.
 exe", co_occurring_keys=["service:dcom"])
135 assert obs1.total_occurrences == 1
136 assert obs1.co_occurrences.get("service:dcom") == 1
137
138 obs2 = memory_storage.record_observation(
139 "process:svchost.exe", co_occurring_keys=["service:dco
 m", "win32:0x80070005"]
140)
141 assert obs2.total_occurrences == 2
142 assert obs2.co_occurrences.get("service:dcom") == 2
143 assert obs2.co_occurrences.get("win32:0x80070005") == 1
144
145 stats = memory_storage.get_stats()
146 assert stats["total_observation_events"] == 2
147
```

\_\_init\_\_.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm Tests - I
 nit
4 # =====
5 # Description:
6 # Модуль реализации компонентов подсистемы Windows AI-Breadb
 oard (__init__).
7 #
8 # Usage Examples:
9 #
10 #
11 # File: __init__.py
12 # Project: ai-breadboard
13 # Package: apps.windows.wikillm.tests
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # Updated: 2026-10-01 13:28:28
17 # =====
18
19 """Модуль реализации компонентов подсистемы Windows AI-Breadbo
 ard (__init__)."""
20
21 # -*- coding: utf-8 -*-
22 # Tests for apps.windows.wikillm
23
```

## tui.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Tui
4 # =====
5 # Description:
6 # Интерактивный консольный терминал для исследования, поиска
7 # и разрешения
8 # Usage Examples:
9 # Python API:
10 # import apps.windows.wikillm.tui as tui
11 #
12 # File: tui.py
13 # Project: ai-breadboard
14 # Package: apps.windows.wikillm
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-01 13:28:28
18 # =====
19
20 from __future__ import annotations
21 """Интерактивный консольный терминал для исследования, поиска
22 и разрешения"""
23
24 import asyncio
25 from typing import Optional
26 from .engine import WikiLLMEngine
27 from .extractor import ArtifactExtractor
28 from .models import ArtifactInput
29
30 async def run_tui(engine: Optional[WikiLLMEngine] = None) -> N
31 one:
```

```

31 | """Запускает интерактивный TUI цикл в консоли."""
32 | eng = engine or WikiLLMEngine()
33 | await eng.start()
34 |
35 | print("\n" + "=" * 70)
36 | print(" WikiLLM: Progressive Windows Knowledge Terminal
37 | ")
38 | print(" Команды: ':stats', ':list', ':q' для выхода")
39 | print("=" * 70 + "\n")
40 | try:
41 | while True:
42 | try:
43 | user_input = input("WikiLLM> ").strip()
44 | except (EOFError, KeyboardInterrupt):
45 | break
46 |
47 | if not user_input:
48 | continue
49 |
50 | if user_input.lower() in (":q", ":quit", "exit"):
51 | break
52 |
53 | if user_input == ":stats":
54 | metrics = eng.get_metrics()
55 | print("\n Статистика базы знаний:")
56 | print(f" Сущностей в БД: {metrics['storage_stats']['total_entities']}")
57 | print(f" Наблюдений артефактов: {metrics['storage_stats']['total_observation_events']}")
58 | print(f" Всего запросов: {metrics['lookup_stats']['total_requests']}")
59 | print(f" L1 Exact Hits: {metrics['lookup_stats']['l1_exact_hits']}")
60 | print(f" L2 Fingerprint Hits: {metrics['lookup_stats']['l2_fingerprint_hits']}")
61 | print(f" L3 Semantic Hits: {metrics['lookup_stats']['l3_semantic_hits']}")

```

```

62 | print(f" L4 Gemini Синтез: {metrics['looku
| p_stats']['l4_gemini_resolutions']}")
63 | print(f" Cache Hit Rate: {metrics['cache
| _hit_rate_pct']}%\n")
64 | continue
65 |
66 | if user_input == ":list":
67 | entities = eng.storage.list_entities(limit=20)
68 | print(f"\n□ Последние {len(entities)} записей
| базы знаний:")
69 | for e in entities:
70 | print(f" • [{e.canonical_key}] {e.name} (
| {e.category})")
71 | print()
72 | continue
73 |
74 | # Разрешение пользовательского запроса
75 | extracted = ArtifactExtractor.from_raw_text(user_i
| nput)
76 | art = extracted[0] if extracted else ArtifactInput
| (raw_query=user_input)
77 |
78 | print(f"[*] Поиск артефакта '{art.raw_query or use
| r_input}'...")
79 | res = await eng.resolve(art, sync_gemini=True)
80 |
81 | print(f"\n[+] Уровень: {res.lookup_level.value.upp
| er()} | Время: {res.execution_time_ms:.1f} мс | Кэш: {res.cach
| ed}")
82 | if res.entity:
83 | print(f" Ключ: {res.entity.canonical_k
| ey}")
84 | print(f" Имя: {res.entity.name}")
85 | print(f" Категория: {res.entity.category} (
| {res.entity.severity})")
86 | print(f" Сводка: {res.entity.summary}")
87 | if res.entity.diagnostic_info and res.entity.d
| iagnostic_info.possible_causes:

```

```
88 | print(f" Причины: {'', ' '.join(res.enti
| ty.diagnostic_info.possible_causes)}")
89 | if res.entity.diagnostic_info and res.entity.d
| iagnostic_info.remediation_steps:
90 | print(" Шаги устранения:")
91 | for s in res.entity.diagnostic_info.remedi
| ation_steps:
92 | print(f" -> {s.title}: {s.descripti
| on}")
93 | else:
94 | print(f" [-] {res.message}")
95 | print()
96 |
97 | finally:
98 | await eng.stop()
99 | print("\nЗавершение работы WikiLLM.")
100 |
```

\_\_init\_\_.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Init
4 # =====
5 # Description:
6 # Инициализатор пакета WikiLLM (Progressive Knowledge Acquisition + Retrieval)
7 #
8 # Usage Examples:
9 #
10 #
11 # File: __init__.py
12 # Project: ai-breadboard
13 # Package: apps.windows.wikillm
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # Updated: 2026-10-01 13:28:28
17 # =====
18
19 """Инициализатор пакета WikiLLM (Progressive Knowledge Acquisition + Retrieval)"""
20
21 from .code_indexer import CodeKnowledgeIndexer
22 from .config import WikiLLMConfig, load_config
23 from .engine import WikiLLMEngine
24 from .extractor import ArtifactExtractor
25 from .models import (
26 ArtifactInput,
27 ArtifactType,
28 Claim,
29 CodeKnowledge,
30 DiagnosticKnowledge,
31 Evidence,
```

```
32 | KnowledgeEntity,
33 | KnowledgeSource,
34 | LookupLevel,
35 | ObservationRecord,
36 | ResolutionAction,
37 | ResolutionResult,
38 |)
39 | from .normalizer import CanonicalKeyNormalizer
40 | from .router import init_router, router
41 | from .storage import WikiStorage
42 | from .telemetry_bridge import TelemetryWikiBridge
43 |
44 | __all__ = [
45 | "WikiLLMEngine",
46 | "WikiStorage",
47 | "WikiLLMConfig",
48 | "load_config",
49 | "ArtifactExtractor",
50 | "CanonicalKeyNormalizer",
51 | "CodeKnowledgeIndexer",
52 | "TelemetryWikiBridge",
53 | "router",
54 | "init_router",
55 | "ArtifactInput",
56 | "ArtifactType",
57 | "KnowledgeEntity",
58 | "KnowledgeSource",
59 | "LookupLevel",
60 | "ResolutionResult",
61 | "ResolutionAction",
62 | "Claim",
63 | "Evidence",
64 | "ObservationRecord",
65 | "DiagnosticKnowledge",
66 | "CodeKnowledge",
67 |]
68 |
```

\_\_main\_\_.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Wikillm - Main
4 # =====
5 # Description:
6 # Точка входа для вызова модуля через py -m apps.windows.wikillm.
7 #
8 # Usage Examples:
9 # CLI:
10 # python -m apps.windows.wikillm.__main__
11 # Python API:
12 # import apps.windows.wikillm.__main__ as __main__
13 #
14 # File: __main__.py
15 # Project: ai-breadboard
16 # Package: apps.windows.wikillm
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """Точка входа для вызова модуля через py -m apps.windows.wikillm."""
23
24 from .cli import main
25
26 if __name__ == "__main__":
27 main()
28
```